



On IND-qCCA Security in the ROM and Its Applications

CPA Security Is Sufficient for TLS 1.3

Loïs Huguenin-Dumittan^(✉) and Serge Vaudenay

EPFL, Lausanne, Switzerland

{lois.huguenin-dumittan, serge.vaudenay}@epfl.ch

Abstract. Bounded IND-CCA security (IND-qCCA) is a notion similar to the traditional IND-CCA security, except the adversary is restricted to a constant number q of decryption/decapsulation queries. We show in this work that IND-qCCA is easily obtained from any passively secure PKE in the (Q)ROM. That is, simply adding a confirmation hash or computing the key as the hash of the plaintext and ciphertext holds an IND-qCCA KEM. In particular, there is no need for derandomization or re-encryption as in the Fujisaki-Okamoto (FO) transform [15]. This makes the decapsulation process of such IND-qCCA KEM much more efficient than its FO-derived counterpart. In addition, IND-qCCA KEMs could be used in the recently proposed KEMTLS protocol [29] that requires IND-1CCA ephemeral key-exchange mechanisms, or in TLS 1.3. Then, using similar proof techniques, we show that CPA-secure KEMs are sufficient for the TLS 1.3 handshake to be secure, solving an open problem in the ROM. In turn, this implies that the PRF-ODH assumption used to prove the security of TLS 1.3 is not necessary and can be replaced by the CDH assumption in the ROM. We also highlight and briefly discuss several use cases of IND-1CCA KEMs in protocols and ratcheting primitives.

1 Introduction

As the NIST standardization process for post-quantum (PQ) public-key cryptography progresses, studying how these new PQ schemes could be integrated into existing protocols has become a hot topic. In particular, the newly adopted TLS 1.3 in its standard form is already “PQ-obsolete” in the sense that only traditional Diffie-Hellman (DH) key-exchange is supported. Indeed, as most PQ schemes come into the form of *Key-Encapsulation Mechanisms* (KEMs) and not Key-Exchange (KEX) such as DH, TLS 1.3 needs modifications to be quantum-resistant.

Several implementations of PQ TLS 1.3 have already been experimented, the most well-known one surely being the OQS-OpenSSL project [1]. This library implements a TLS handshake that supports KEMs and *hybrid cryptography* (i.e. the final shared secret is a combination of a DH secret and a KEM secret/key).

The changes compared to the standard version of the TLS 1.3 handshake are minimal. That is, the client (resp. server) DH share is replaced by a public-key (resp. a ciphertext encapsulated under the public-key), and the shared secret is the key encapsulated in the ciphertext. Several works have analysed the performance and implementation challenges of OQS-OpenSSL (e.g. [9, 25]).

More recently, based on the observation that (PQ) KEM public-keys/ciphertexts are usually more compact than (PQ) public-keys/signatures, Schwabe et al. [29] proposed KEMTLS as a variant of the TLS 1.3 handshake. The main difference between both protocols is that KEMTLS uses a KEM for (implicit) server authentication instead of a signature. This reduces the overall bandwidth of the handshake and the computation time on the server-side. Thus, two KEMs are used in KEMTLS: one for establishing an ephemeral shared secret and the other one to authenticate the server. While the latter needs to be IND-CCA secure as it uses long-term keys, the authors showed that IND-1CCA security is sufficient for the former KEM for the whole handshake to be secure. That is, the KEM needs to be secure against an adversary that can make a *unique* decapsulation query. Similarly, in the security proof of TLS 1.3 handshake by Dowling et al. [12], DH key-exchange can be replaced by an IND-1CCA KEM and the proof would still go through.

However, in KEMTLS or PQ implementations of TLS 1.3 (e.g. [1]), the ephemeral KEMs are implemented with IND-CCA KEMs, which are usually obtained by applying the Fujisaki-Okamoto (FO) transform or a variant (e.g. [15, 19]) on an OW/IND-CPA public-key encryption scheme (PKE). The FO construction re-encrypts the decrypted plaintext during decapsulation, making it an expensive operation. This motivates the present work, which studies whether IND-1CCA KEM can be obtained from CPA-secure PKEs through a more efficient transform than FO (in the ROM). We reply by the affirmative by showing that IND-1CCA KEMs with much faster decapsulation than FO-derived IND-CCA KEM can be obtained from any CPA-secure PKE. Using similar tools, we also study the security of the PQ TLS 1.3 handshake when the KEM used for key exchange is only CPA-secure.

Our Contributions

We show how to build an efficient IND-qCCA KEM (i.e. the adversary can only make q decapsulation queries) from any OW-CPA PKE in the ROM. The bound has a loose factor of 2^q , making it insecure or impractical for large q . However, such construction is sufficient to build an efficient IND-1CCA KEM from any OW-CPA public-key encryption scheme. The transform simply sends a confirmation hash along the ciphertext encrypting the seed. In addition, we prove the security of this construction in the QROM as well.

Such a transform might be useful in several applications such as the KEMTLS protocol [29] mentioned above, PQ variants of TLS 1.3 or ratcheting, as discussed in Sect. 5.

Similarly, we show that deriving the key as $K := H(m, ct)$, where m is the seed encrypted in the ciphertext ct , holds an IND-qCCA KEM in the ROM. The

bound is worse compared to the first transform, having a $\approx q_H^{2q}$ factor, where q_H is the number of queries an adversary can make to the random oracle H . The intuition is that any decapsulation query that returns $H(m, \text{ct})$ with $\text{ct} \neq \text{ct}^*$ does not help much the adversary to recover the real key $H(m^*, \text{ct}^*)$ due to the independence of RO values. However, each query to the decapsulation oracle still leaks a little information (such as equality between decrypted values), leading to the $\approx q_H^{2q}$ factor.

Compared to the FO transform and its variants, our CPA-to-qCCA transforms offer several advantages. The main one is a significant speed boost in decapsulation, as there is no need for re-encryption. Depending on the cost of encryption of the underlying scheme, the difference can be large. For instance, removing the re-encryption check in the optimized version of the isogeny-based scheme SIKE [21] cuts by more than 50% the decapsulation time (32235377 vs 73282449 cycles for SIKEp434_compressed on Ubuntu 21.04 with 2.8GHz Intel Core i7-1165G7). Another interesting feature of our transform is that we do not need to de-randomize the encryption (i.e. computing the random coins for encapsulation as the hash of the message/seed), removing the need for an additional random oracle.

We then consider the PQ TLS 1.3 handshake as it is implemented in OQS-OpenSSL [1]. Based on the observation that the key-schedule computes the keys as key-derivation functions (KDFs) applied on the shared secret and (the hash of) the transcript so far (including the ciphertext), we prove that if the KEM is OW-CPA secure, then the handshake is secure in the MultiStage model of Dowling et al. [12]. The proof is inspired by the proof of security of our second transform. Note that this result holds in the ROM (the KDFs/hash function are assumed to be ROs) and the security bound is very much “non-tight”. Still, this shows that CPA-secure KEMs are sufficient for the TLS 1.3 handshake to be secure, solving an open problem raised by several authors (e.g. [12, 25]). Then, since one can consider DH as a KEM, this implies that TLS 1.3 is secure as long as the *computational Diffie-Hellman* (CDH) problem is hard, showing that the PRF-ODH assumption used in the original proof [12] is not necessary (in the ROM). We note that this last result can also be derived from the fact that DH as used in TLS 1.3 is a IND-1CCA KEM in the ROM, assuming that CDH is hard. We prove this in the full version of the paper [20].

Finally, in Sect. 5, we discuss possible use cases of IND-qCCA in the context of communication protocols and ratcheting primitives. In particular, we note that IND-1CCA security is sufficient in many recent applications as the trend is to move to forward secure schemes, which discard key pairs after one use.

Remark on IND-CPA vs IND-1CCA

We note that plain IND-CPA PQ schemes are often not IND-1CCA. In particular, it is stated in Sect. 4.3 of the KEMTLS paper [30]:

“We leave as an open question to what extent non-FO-protected post-quantum KEMs may be secure against a single decapsulation query, but at this point IND-CCA is the safe choice.”

The answer to this question obviously depends on how the “non-FO protected” IND-CPA PKE is used as a KEM. However, if it is used in the trivial way (i.e. $m \leftarrow \mathcal{M}$, $K := H(m)$, $\text{ct} := \text{enc}(\text{pk}, m)$), the resulting KEM can usually be broken with 1 query for most of the PQ schemes. The adversary receives K^* , $\text{ct}^* := \text{enc}(\text{pk}, m^*)$, queries $\text{ct}^* + \delta$ and gets back $H(m^*)$ with high probability, if δ is “small”. Then, it can just compare whether $H(m^*) = K^*$ or not and break IND-1CCA security. The reaction attacks (e.g. [13]) requiring thousands of queries mentioned in the same paper [30] are key-recovery attacks, not distinguishing attacks. The simple distinguishing adversary given above actually gives a good intuition of why adding a confirmation hash $H'(m, \text{ct})$ along the ciphertext as in our first transform holds a IND-qCCA KEM. In order to submit a valid decapsulation query, the adversary must compute $H'(m, \text{ct})$ with $\text{ct} \neq \text{ct}^*$. Hence, the adversary itself needs to query $H'(m, \text{ct})$ beforehand, thus it knows m and the decapsulation query is (nearly) useless.

Related Work

The notion of bounded IND-CCA (i.e. IND-qCCA) has been studied in several works. Cramer et al. [8] defined IND-qCCA and showed that one can build an IND-qCCA PKE from any CPA-secure PKE in a black-box manner in the standard model, using one-time signatures. While this construction is valid in the standard model and ours in the ROM only, their reduction is inefficient compared to FO transforms, which we aim to improve. Following their work, Peirera et al. [26] built a more efficient IND-qCCA PKE based on the CDH assumption, and Yamakawa et al. [32] proposed other constructions based on the factoring and bilinear CDH assumptions. As far as we know, we are the first to note that a IND-qCCA KEM can be obtained from any CPA-secure PKE through a very simple and efficient transform in the ROM.

Starting from the original Fujisaki-Okamoto transform [14, 15], many works have been dedicated to building variants of FO with tighter security bounds in the QROM (e.g. [4, 19, 23, 28]). While these are CPA-to-CCA transforms, ours guarantee qCCA security only but at a lesser computational cost.

Dowling et al. [12] proved the security of the standard TLS 1.3 handshake in the MultiStage security model. We extend their result by showing that TLS 1.3 security still holds if the DH KEX is replaced by a CPA-secure KEM (in the ROM). In turn, this also implies that the CDH assumption is sufficient for proving the security of the original TLS 1.3, which was based on the PRF-ODH assumption so far. In two more recent works, Diemert et al. [11] and Davis et al. [10] aimed at proving a tighter security bound for TLS 1.3. Their proofs are valid in the ROM and are based on the Strong Diffie-Hellman (SDH) assumption. Our result on TLS 1.3 is complementary to theirs in the sense that we prove that TLS security holds under a weaker assumption but with a looser security bound.

Brendel et al. [5] studied the PRF-ODH assumption. In particular, they showed that PRF-ODH is hard if the SDH assumption holds in the ROM. The PRF-ODH notion considered in their work is generic as the adversary can query

two types of “decapsulation” oracles multiple times. On the other hand, if we restrict ourselves to the notion where the adversary can make a unique query (which is sufficient for TLS 1.3 security), we show in the full version of the paper [20] that CDH hardness is sufficient for PRF-ODH (with one query) to hold.

Finally, following the KEMTLS paper [29], several recent works used the notion of IND-1CCA KEM to build secure protocols (e.g. [6, 18, 31]), showing the growing importance of such a notion.

2 Preliminaries

2.1 Notation

For $\mathcal{A}$ a randomized algorithm, we write $b \leftarrow_{\$} \mathcal{A}$ to indicate b is set to the value output by $\mathcal{A}$. Similarly, if Ψ (resp. $\mathcal{X}$) is a distribution (resp. a set), then $x \leftarrow_{\$} \Psi$ (resp. $x \leftarrow_{\$} \mathcal{X}$) means that x is sampled uniformly at random from Ψ (resp. $\mathcal{X}$). We denote by 1_P the indicator function which returns 1 if the predicate P is fulfilled and 0 otherwise. We write $[n]$ the set $\{1, \dots, n\}$. For $\mathcal{A}$ an algorithm, we write $\mathcal{A} \Rightarrow b$ to denote the event $\mathcal{A}$ outputs b . Finally, in a game, we write **abort** to mean that the algorithm is stopped.

2.2 Public-Key Encryption Scheme

A Public-Key Encryption (PKE) scheme is defined as follows.

Definition 1 (Public-Key Encryption). *A Public-Key Encryption scheme over a domain $\mathcal{M}$ is composed of three algorithms $\text{gen}, \text{enc}, \text{dec}$:*

- $(\text{pk}, \text{sk}) \leftarrow_{\$} \text{gen}(1^\lambda)$: *The key generation algorithm takes the security parameter as input and outputs the public key pk and the secret key sk .*
- $\text{ct} \leftarrow_{\$} \text{enc}(\text{pk}, \text{pt})$: *The encryption algorithm takes as inputs the public key pk and a plaintext $\text{pt} \in \mathcal{M}$ and it outputs a ciphertext ct .*
- $\text{pt}' \leftarrow \text{dec}(\text{sk}, \text{ct})$: *The decryption procedure takes as inputs the secret key sk and the ciphertext $\text{ct} \in \mathcal{C}$ and it outputs a plaintext $\text{pt}' \in \mathcal{M} \cup \{\perp\}$.*

The gen and enc are probabilistic algorithms that can be made deterministic by adding random coins as inputs. The decryption procedure is deterministic.

Correctness. We say a PKE scheme is δ correct if for any ppt adversary $\mathcal{A}$ playing the game CORR defined in Fig. 1, we have

$$\Pr[\text{CORR}_{\text{PKE}}(\mathcal{A}) \Rightarrow 1] \leq \delta(\lambda)$$

where λ is the security parameter, we omit it from now on for the sake of simplicity.

$\text{CORR}_{\text{PKE}}(\mathcal{A})$

```

(pk, sk)  $\leftarrow$   $\$$  gen( $1^\lambda$ )
pt  $\leftarrow$   $\mathcal{A}$ (pk, sk)
ct  $\leftarrow$   $\$$  enc(pk, pt)
return  $1_{\text{dec}(\text{sk}, \text{ct}) \neq \text{pt}}$ 

```

Fig. 1. Correctness game.

$\text{OW-ATK}_{\text{PKE}}(\mathcal{A})$ <pre> (pk, sk) $\leftarrow$ $\\$ gen(1^λ) pt* $\leftarrow$ $\\$ $\mathcal{M}$ ct* $\leftarrow$ enc(pk, pt*) pt' $\leftarrow$ $\mathcal{A}^{\mathcal{O}^{\text{ATK}}}(\text{pk}, \text{ct}^*)$ return $1_{\text{pt}' = \text{pt}^*}$ </pre>	Oracle $\mathcal{O}^{\text{PCO}}(\text{pt}, \text{ct})$ <pre> pt' $\leftarrow$ dec(sk, ct) return $1_{\text{pt}' = \text{pt}}$ </pre>	<table border="1" style="width: 100%; text-align: center; border-collapse: collapse;"> <tr> <td style="padding: 2px 10px;">ATK</td> <td style="padding: 2px 10px;">CPA</td> <td style="padding: 2px 10px;">PCA</td> </tr> <tr> <td style="padding: 2px 10px;">$\mathcal{O}^{\text{ATK}}$</td> <td style="padding: 2px 10px;">$\perp$</td> <td style="padding: 2px 10px;">$\mathcal{O}^{\text{PCO}}$</td> </tr> </table>	ATK	CPA	PCA	$\mathcal{O}^{\text{ATK}}$	$\perp$	$\mathcal{O}^{\text{PCO}}$
ATK	CPA	PCA						
$\mathcal{O}^{\text{ATK}}$	$\perp$	$\mathcal{O}^{\text{PCO}}$						

Fig. 2. One-Wayness games.

Plaintext Checking. We recall the notions of One-Wayness under Chosen Plaintext Attacks (OW-CPA) and Plaintext-Checking Attacks (OW-PCA).

Definition 2 (One-Wayness and Plaintext Checking). *Let $\mathcal{M}$ be a finite message space, PKE a PKE scheme over $\mathcal{M}$ and we consider the games defined on the left in Fig. 2 with the different oracles as defined in the table on the right of Fig. 2. Then, PKE is OW-ATK, for $\text{ATK} \in \{\text{CPA}, \text{PCA}\}$, if for any ppt adversary $\mathcal{A}$ we have*

$$\text{Adv}_{\text{PKE}}^{\text{ow-atk}}(\mathcal{A}) = \Pr[\text{OW-ATK}_{\text{PKE}}(\mathcal{A}) \Rightarrow 1] = \text{negl}(\lambda)$$

where $\Pr[\text{OW-ATK}_{\text{PKE}}(\mathcal{A}) \Rightarrow 1]$ is the probability that the adversary wins the OW-ATK game.

2.3 Key Encapsulation Mechanism (KEM)

A Key Encapsulation Mechanism is defined as follows.

Definition 3 (Key Encapsulation Mechanism). *A KEM over $\mathcal{K}$ is a tuple of three algorithms gen, encaps, decaps:*

- $(\text{pk}, \text{sk}) \leftarrow \$ \text{gen}(1^\lambda)$: The key generation algorithm takes as inputs the security parameter and it outputs the public key pk and the secret key sk .
- $\text{ct}, K \leftarrow \$ \text{encaps}(\text{pk})$: The encapsulation algorithm takes as inputs the public key pk and it outputs a ciphertext $\text{ct} \in \mathcal{C}$ and a key $K \in \mathcal{K}$.
- $K' \leftarrow \text{decaps}(\text{sk}, \text{ct})$: The decapsulation procedure takes as inputs the secret key sk and the ciphertext $\text{ct} \in \mathcal{C}$ and it outputs a key K . If the KEM allows explicit rejection, the output is a key $K \in \mathcal{K}$ or the rejection symbol $\perp$. If the rejection is implicit, the output is always a key $K \in \mathcal{K}$.

IND-(q)CCA _{KEM} ($\mathcal{A}$)	Oracle $\mathcal{O}^{\text{Dec}}(\text{ct})$
$(\text{pk}, \text{sk}) \leftarrow \text{gen}(1^\lambda)$	if $\text{ct} = \text{ct}^* : \text{return } \perp$
$b \leftarrow \{0, 1\}$	if more than q queries : return $\perp$ // If IND-qCCA
$\text{ct}^*, K_0 \leftarrow \text{encaps}(\text{pk})$	$K' \leftarrow \text{decaps}(\text{sk}, \text{ct})$
$K_1 \leftarrow \mathcal{K}$	return K'
$b' \leftarrow \mathcal{A}^{\mathcal{O}^{\text{Dec}}}(\text{pk}, \text{ct}^*, K_b)$	
return $1_{b'=b}$	

Fig. 3. Indistinguishability games.

The gen and encaps are probabilistic algorithms. The randomness can be made explicit by adding random coins as inputs. The decapsulation function is deterministic.

Indistinguishability Security. KEM indistinguishability is defined as follows.

Definition 4 (KEM Indistinguishability). We consider the games defined in Fig. 3. Let $\mathcal{K}$ be a finite key space. A KEM scheme over $\mathcal{K}$ $\text{KEM} = (\text{gen}, \text{encaps}, \text{decaps})$ is IND-CCA (resp. IND-qCCA) if for any ppt adversary $\mathcal{A}$ (resp. any ppt $\mathcal{A}$ limited to q decapsulation queries) we have

$$\text{Adv}_{\text{KEM}}^{\text{ind-(q)cca}}(\mathcal{A}) = \left| \Pr [\text{IND} - (\text{q})\text{CCA}_{\text{KEM}}(\mathcal{A}) \Rightarrow 1] - \frac{1}{2} \right| = \text{negl}(\lambda)$$

where $\Pr [\text{IND} - (\text{q})\text{CCA}_{\text{KEM}}(\mathcal{A}) \Rightarrow 1]$ is the probability that $\mathcal{A}$ wins the IND-(q)CCA_{KEM}($\mathcal{A}$) game defined in Fig. 3.

We can also define OW-CPA for KEMs, which is similar to the equivalent notion for PKE.

Definition 5 (KEM OW-CPA). A KEM scheme $\text{KEM} = (\text{gen}, \text{encaps}, \text{decaps})$ is OW-CPA if for any ppt adversary $\mathcal{A}$ we have

$$\begin{aligned} \text{Adv}_{\text{KEM}}^{\text{ow-cpa}}(\mathcal{A}) &= \Pr [\mathcal{A}(\text{pk}, \text{ct}^*) \Rightarrow K : (\text{pk}, \text{sk}) \leftarrow \text{gen}(1^\lambda); (K, \text{ct}^*) \leftarrow \text{encaps}(\text{pk})] \\ &= \text{negl}(\lambda) , \end{aligned}$$

where the probability is taken over the randomness of the public-key generation, encapsulation and the adversary $\mathcal{A}$.

<u>gen()</u>	<u>encaps(pk)</u>	<u>decaps(sk, ct)</u>
$(pk, sk) \leftarrow \$ \text{gen}^P()$	$\sigma \leftarrow \$ \mathcal{M}$	$(ct'_0, \text{tag}') \leftarrow \text{ct}$
return (pk, sk)	$ct_0 \leftarrow \$ \text{enc}^P(pk, \sigma)$	$\sigma' \leftarrow \text{dec}^P(sk, ct'_0)$
	$\text{tag} \leftarrow H'(\sigma, ct_0)$	if $H'(\sigma', ct'_0) \neq \text{tag}'$:
	$K \leftarrow H(\sigma)$	return $\perp$
	return $K, (ct_0, \text{tag})$	return $H(\sigma')$

Fig. 4. T_{CH} transform.

3 OW-CPA to IND-qCCA Transforms

We first prove the following simple lemma.

Lemma 1. *Let PKE be a PKE. Then, for any ppt OW-PCA adversary $\mathcal{A}$ making at most q queries to the PCO oracle, there exists a OW-CPA adversary $\mathcal{B}$ s.t.*

$$\text{Adv}_{\text{PKE}}^{\text{ow-pca}}(\mathcal{A}) \leq 2^q \cdot \text{Adv}_{\text{PKE}}^{\text{ow-cpa}}(\mathcal{B}).$$

Proof. We can simply see that the PCO oracle returns 1 bit of information, thus PKE loses at most q bits of security when a PCO oracle is available. More formally, given $\mathcal{A}$, one can build $\mathcal{B}$ as follows. It passes its input to $\mathcal{A}$ and simulates the PCO oracle by sampling a response at random in $\{0, 1\}$. Then, it returns the response of $\mathcal{A}$. Its probability of success is $\text{Adv}_{\text{PKE}}^{\text{ow-cpa}}(\mathcal{B}) \geq \frac{1}{2^q} \text{Adv}_{\text{PKE}}^{\text{ow-pca}}(\mathcal{A})$, as the probability the q responses are correct is $\frac{1}{2^q}$. $\square$

We consider the transform T_{CH} given in Fig. 4. This construction takes a PKE $\text{PKE} = (\text{gen}^P, \text{enc}^P, \text{dec}^P)$ and outputs a KEM $(\text{gen}, \text{encaps}, \text{decaps})$. Note that T_{CH} is basically the REACT transform [24] without the asymmetric part (to get a KEM instead of a PKE).

We now show that the resulting KEM is IND-qCCA assuming the underlying PKE is OW-PCA.

Theorem 1. *We consider two random oracles $H, H' : \{0, 1\}^* \mapsto \{0, 1\}^n$. Let KEM be the KEM resulting from applying the T_{CH} transform to a δ -correct PKE. Then, for any IND-qCCA adversary $\mathcal{A}$ that makes at most q_H (resp. $q_{H'}$) queries to H (resp. H'), there exists a OW-PCA adversary $\mathcal{B}$ s.t.*

$$\text{Adv}_{\text{KEM}}^{\text{ind-qcca}}(\mathcal{A}) \leq \frac{(q + q_{H'} + 1)^2}{2^n} + \delta + \frac{q}{2^n} + (q_H + q_{H'}) \cdot \text{Adv}_{\text{PKE}}^{\text{ow-pca}}(\mathcal{B}),$$

where $\mathcal{B}$ makes at most q queries to its plaintext-checking oracle. In addition, if PKE is a deterministic encryption scheme, the bound becomes

$$\text{Adv}_{\text{KEM}}^{\text{ind-qcca}}(\mathcal{A}) \leq \frac{(q + q_{H'} + 1)^2}{2^n} + \delta + \frac{q}{2^n} + \text{Adv}_{\text{PKE}}^{\text{ow-pca}}(\mathcal{B}).$$

$\Gamma^{0-3}(\mathcal{A})$	Oracle $\mathcal{O}^{\text{Dec}}(\text{ct})$	Oracle $\mathcal{O}^{\text{Dec2}}(\text{ct})$
$(\text{pk}, \text{sk}) \leftarrow \text{gen}$ $b \leftarrow \{0, 1\}$ $\sigma^* \leftarrow \{0, 1\}^n$ $\text{ct}_0^* \leftarrow \text{enc}^p(\text{pk}, \sigma^*)$ $K_0 \leftarrow H(\sigma^*); h^* \leftarrow H'(\sigma^*, \text{ct}_0^*)$ $K_1 \leftarrow \mathcal{K}$ $\text{ct}^* \leftarrow (\text{ct}_0^*, h^*)$ $b' \leftarrow \mathcal{A}^{\mathcal{O}^{\text{Dec}}}(\text{pk}, \text{ct}^*, K_b) \parallel \Gamma^0\text{-}\Gamma^1$ $b' \leftarrow \mathcal{A}^{\mathcal{O}^{\text{Dec2}}}(\text{pk}, \text{ct}^*, K_b) \parallel \Gamma^2\text{-}\Gamma^3$ if query : abort $\parallel \Gamma^3$ return $1_{b'=b}$	if $\text{ct} = \text{ct}^* :$ return $\perp$ if more than q queries : return $\perp$ $(\text{ct}_0, h) \leftarrow \text{ct}$ if $\text{ct}_0 = \text{ct}_0^*$ or $h = h^* : \parallel \Gamma^1\text{-}\Gamma^3$ return $\perp \parallel \Gamma^1\text{-}\Gamma^3$ $\sigma' \leftarrow \text{dec}^p(\text{sk}, \text{ct}_0')$ if $H'(\sigma', \text{ct}_0) \neq h :$ return $\perp$ return $H(\sigma')$	if $\text{ct} = \text{ct}^* :$ return $\perp$ if more than q queries : return $\perp$ $(\text{ct}_0, h) \leftarrow \text{ct}$ if $\text{ct}_0 = \text{ct}_0^*$ or $h = h^* :$ return $\perp$ if $\exists \sigma$ s.t. $((\sigma, \text{ct}_0), h) \in \mathcal{L}_{H'} :$ if $\mathcal{O}^{\text{PCO}}(\sigma, \text{ct}_0) :$ return $H(\sigma)$ return $\perp$
$H(\sigma)$	$H'(\sigma, \text{ct})$	
if $\exists h$ s.t. $(\sigma, h) \in \mathcal{L}_H :$ return h if $\sigma = \sigma^* :$ query $\leftarrow$ true $\parallel \Gamma^3$ $h \leftarrow \{0, 1\}^n$ $\mathcal{L}_H \leftarrow \mathcal{L}_H \cup \{(\sigma, h)\}$ return h	if $\exists h$ s.t. $((\sigma, \text{ct}), h) \in \mathcal{L}_{H'} :$ return h if $\sigma = \sigma^* :$ query $\leftarrow$ true $\parallel \Gamma^3$ $h \leftarrow \{0, 1\}^n$ $\mathcal{L}_{H'} \leftarrow \mathcal{L}_{H'} \cup \{((\sigma, \text{ct}), h)\}$ if $\exists x, x', h$ s.t. $x \neq x'$ $\wedge (x, h) \in \mathcal{L}_{H'}$ $\wedge (x', h) \in \mathcal{L}_{H'} :$ abort return h	

Fig. 5. Sequence of games for the proof of Theorem 1. $\mathcal{O}^{\text{PCO}}$ is defined as in the OW-PCA game (see Fig. 2).

Proof. We proceed by game hopping, the sequence of games is presented in Fig. 5. Let $\mathcal{L}_H$ (resp. $\mathcal{L}_{H'}$) be the list of queries (x, h) made to the RO H (resp. H') s.t. $H(x) = h$ (resp. $H'(x) = h$). In addition, let the challenge ciphertext be $\text{ct}^* = (\text{ct}_0^*, h^*)$, and σ^* be s.t. $\text{enc}^p(\text{pk}, \sigma^*) = \text{ct}_0^*$. We start with game Γ^0 which is the IND-qCCA game, except we abort if the adversary finds a collision on H' (i.e. $H'(x) = H'(x')$ for $x \neq x'$ and $(x, h), (x', h) \in \mathcal{L}_{H'}$). This happens with prob. at most $\frac{(q+q_{H'}+1)^2}{2^n}$ and we have

$$|\Pr[\text{IND-qCCA}_{\text{KEM}}(\mathcal{A}) \Rightarrow 1] - \Pr[\Gamma^0(\mathcal{A}) \Rightarrow 1]| \leq \frac{(q + q_{H'} + 1)^2}{2^n}.$$

Γ^1 : The decapsulation oracle is modified s.t. it returns $\perp$ whenever ct_0^* or h^* is queried (note that both cannot be submitted at the same time). This game is the same as Γ^0 except if the oracle in Γ^0 does not return $\perp$ on such queries. Let bad be this event. We split this into two cases:

- $\mathcal{O}^{\text{Dec}}(\text{ct}_0^*, h \neq h^*) \neq \perp$. This happens only if

$$H'(\text{dec}(\text{sk}, \text{ct}_0^*), \text{ct}_0^*) = h \neq h^* = H'(\sigma^*, \text{ct}_0^*).$$

In turn, this implies that $\text{dec}(\text{sk}, \text{ct}_0^*) \neq \sigma^*$ and thus it is a correctness error. Such an error happens at most with probability δ .

- $\mathcal{O}^{\text{Dec}}(\text{ct}_0 \neq \text{ct}_0^*, h^*) \neq \perp$. It means that $h^* = H'(\sigma^*, \text{ct}_0^*) = H'(\sigma', \text{ct}_0)$, with $\sigma' \leftarrow \text{dec}^P(\text{sk}, \text{ct}_0)$, which is not possible since $\text{ct}_0 \neq \text{ct}_0^*$ and we assume no collision occurs.

Therefore, overall $\Pr[\text{bad}] \leq \delta$ and

$$|\Pr[\Gamma^0 \Rightarrow 1] - \Pr[\Gamma^1 \Rightarrow 1]| \leq \Pr[\text{bad}] \leq \delta.$$

Γ^2 : We modify the decapsulation oracle into another oracle $\mathcal{O}^{\text{Dec}^2}$ as follows. On a decapsulation query (ct_0, h) (with $\sigma' \leftarrow \text{dec}(\text{sk}, \text{ct}_0)$):

1. If there is no $((\sigma, \text{ct}_0), h)$ in $\mathcal{L}_{H'}$: return $\perp$. This differs from the previous game only if $h = H'(\sigma', \text{ct}_0)$ but (σ', ct_0) was never queried to H' . As the RO values are uniformly distributed, this happens at most with probability $\frac{1}{2^n}$.
2. If $((\sigma, \text{ct}_0), h) \in \mathcal{L}_{H'}$ for some σ : If $\mathcal{O}^{\text{PCO}}(\sigma, \text{ct}_0) := 1_{\text{dec}(\text{sk}, \text{ct}_0) = \sigma} = 1$, return $H(\sigma)$. Otherwise, return $\perp$. This perfectly simulates the previous oracle as $\mathcal{O}^{\text{PCO}}(\sigma, \text{ct}_0) = 1$ iff $\sigma = \sigma'$ and we know $h = H(\sigma = \sigma', \text{ct}_0)$.

Note that there is at most one σ s.t. $((\sigma, \text{ct}_0), h) \in \mathcal{L}_{H'}$ as we assume no collision occurs. In particular, it means that $\mathcal{O}^{\text{PCO}}$ is called at most once every decapsulation query.

Therefore, by a union bound we get

$$|\Pr[\Gamma^1 \Rightarrow 1] - \Pr[\Gamma^2 \Rightarrow 1]| \leq \frac{q}{2^n}.$$

Γ^3 : Finally, we abort whenever $\mathcal{A}$ queries σ^* to H or $(\sigma^*, \cdot)$ to H' . Let this event be **query**. Note that $\mathcal{A}$ could also learn the value of $H(\sigma^*)$ through a query to $\mathcal{O}^{\text{Dec}^2}$. However, the latter oracle would return $H(\sigma^*)$ only if $\mathcal{A}$ queried $H'(\sigma^*, \cdot)$ before (thus triggering **query**).

Then, we can build a OW-PCA adversary $\mathcal{B}$ (shown in Fig. 6) that perfectly simulates $\mathcal{A}$'s view as long as **query** does not happen. More precisely, $\mathcal{B}$ can simulate the decapsulation oracle using its PCO oracle. Then, on input $(\text{pk}, \text{ct}_0^*)$, $\mathcal{B}$ runs $\mathcal{A}(\text{pk}, (\text{ct}_0^*, h^*), K^*)$, where h^* and K^* are picked at random. Unless **query** occurs, $\mathcal{A}$ cannot distinguish between these random h^*, K^* and the real ones. Finally, if **query** occurs, $\mathcal{B}$ can recover σ^* with probability $\frac{1}{q_H + q_{H'}}$ by sampling a random σ from $S = \{\sigma : (\sigma, *) \in \mathcal{L}_H^A \vee ((\sigma, *), *) \in \mathcal{L}_{H'}\}$, where $\mathcal{L}_H^A$ is the set of queries to H made by $\mathcal{A}$. Thus,

$$|\Pr[\Gamma^2 \Rightarrow 1] - \Pr[\Gamma^3 \Rightarrow 1]| \leq \Pr[\text{query}] \leq (q_H + q_{H'}) \cdot \text{Adv}_{\text{PKE}}^{\text{ow-pca}}(\mathcal{B})$$

where $\mathcal{B}$ makes q query to the PCO oracle. Note that if PKE is deterministic, $\mathcal{B}$ can check whether $\text{enc}(\text{pk}, \sigma) = \text{ct}_0^*$ for all $\sigma \in S$ to find σ^* . This fails only if there exists $\sigma' \neq \sigma^*$ s.t. $\text{enc}(\text{pk}, \sigma') = \text{ct}_0^*$. In turn this implies that there exists $\sigma \in S \cup \{\sigma^*\}$ that would break the correctness, but such an event is already covered by the previous δ factor. In this case, we obtain

$$|\Pr[\Gamma^2 \Rightarrow 1] - \Pr[\Gamma^3 \Rightarrow 1]| \leq \Pr[\text{query}] \leq \text{Adv}_{\text{PKE}}^{\text{ow-pca}}(\mathcal{B}).$$

$\mathcal{B}^{\mathcal{O}^{\text{PCO}}}(\text{pk}, \text{ct}^*)$	Oracle $\mathcal{O}^{\text{Dec2}}(\text{ct})$
init $\mathcal{L}_H, \mathcal{L}_{H'} \leftarrow \emptyset$	if more than q queries :
$h^* \leftarrow \$ \{0, 1\}^n$	return $\perp$
$K^* \leftarrow \$ \{0, 1\}^n$	$(\text{ct}_0, h) \leftarrow \text{ct}$
simulate H, H' for $\mathcal{A}$ with lazy sampling:	if $\text{ct}_0 = \text{ct}_0^*$ or $h = h^*$:
run $\mathcal{A}^{H, H', \mathcal{O}^{\text{Dec2}}}(\text{pk}, (\text{ct}^*, h^*), K^*)$	return $\perp$
$\sigma' \leftarrow \$ \{\sigma : \sigma \in \mathcal{L}_H^A \vee (\sigma, *) \in \mathcal{L}_{H'}\}$	if $\exists \sigma$ s.t. $((\sigma, \text{ct}), h) \in \mathcal{L}_{H'}$:
return σ'	if $\mathcal{O}^{\text{PCO}}(\sigma, \text{ct}_0)$: return $H(\sigma)$
	return $\perp$

Fig. 6. $\mathcal{B}$ adversary for the proof of Theorem 1.

Finally, since $\mathcal{A}$ cannot query σ^* to H anymore, it cannot distinguish between a random key and $H(\sigma^*)$. Hence, $\Pr[\Gamma^3 \Rightarrow 1] = \frac{1}{2}$. Collecting the probabilities holds the result. $\square$

Corollary 1. *We consider two random oracles $H, H' : \{0, 1\}^* \mapsto \{0, 1\}^n$. Let KEM be the KEM resulting from applying the T_{CH} transform to a δ -correct PKE. Then, for any IND-qCCA adversary $\mathcal{A}$ that makes at most q_H (resp. $q_{H'}$) queries to H (resp. H'), there exists a OW-CPA adversary $\mathcal{B}$ s.t.*

$$\text{Adv}_{\text{KEM}}^{\text{ind-qcca}}(\mathcal{A}) \leq \frac{(q + q_{H'})^2}{2^n} + \delta + \frac{q}{2^n} + (q_H + q_{H'} + q)2^q \cdot \text{Adv}_{\text{PKE}}^{\text{ow-cpa}}(\mathcal{B}).$$

If PKE is deterministic, we get

$$\text{Adv}_{\text{KEM}}^{\text{ind-qcca}}(\mathcal{A}) \leq \frac{(q + q_{H'})^2}{2^n} + \delta + \frac{q}{2^n} + 2^q \cdot \text{Adv}_{\text{PKE}}^{\text{ow-cpa}}(\mathcal{B}).$$

In particular, in the case of IND-1CCA (i.e. $q = 1$), if the underlying PKE is OW-CPA the KEM obtained from the T_{CH} transform is IND-1CCA with a security loss of ≈ 1 bit compared to the OW-CPA advantage (if we omit the other negligible terms). Finally, we note that as q is a constant that does not depend on the security parameter (e.g. n) of the PKE, if the OW-CPA advantage of the PKE is negligible, so is the KEM IND-qCCA one. However, in practice, we would need to take n very large to guarantee security for more than a few queries.

<u>gen()</u>	<u>encaps(pk)</u>	<u>decaps(sk, ct)</u>
$(pk, sk) \leftarrow \text{gen}^p()$	$\sigma \leftarrow \text{encaps}(\mathcal{M})$	$\sigma' \leftarrow \text{decaps}^p(sk, ct)$
return (pk, sk)	$ct \leftarrow \text{encaps}^p(pk, \sigma)$	if $\sigma' = \perp$: return $\perp$
	$K \leftarrow H(\sigma, ct)$	return $H(\sigma', ct)$
	return K, ct	

Fig. 7. T_H transform.

3.1 Security in the QROM

We also show that the T_{CH} transform is secure in the Quantum Random Oracle Model (QROM) by proving that Theorem 1 holds in the QROM.

Theorem 2. *We consider two quantum random oracles $H, H' : \{0, 1\}^* \mapsto \{0, 1\}^n$. Let KEM be the KEM resulting from applying the T_{CH} transform to a PKE. Then, for any IND-qCCA adversary $\mathcal{A}$ that makes at most q_H (resp. $q_{H'}$) quantum queries to H (resp. H'), there exists a OW-PCA adversary $\mathcal{B}$ s.t.*

$$\text{Adv}_{\text{KEM}}^{\text{ind-qcca}}(\mathcal{A}) \leq \delta + 2(2q_{H'} + q_H + q) \cdot \sqrt{(2(2q_{H'} + q))^q \cdot \text{Adv}_{\text{PKE}}^{\text{ow-pca}}(\mathcal{B})}$$

where $\mathcal{B}$ makes at most q queries to its plaintext-checking oracle.

Proof. Due to space constraint, we defer the proof to the full version of the paper [20]. $\square$

Corollary 2. *We consider two quantum random oracles $H, H' : \{0, 1\}^* \mapsto \{0, 1\}^n$. Let KEM be the KEM resulting from applying the T_{CH} transform to a δ -correct PKE. Then, for any IND-qCCA adversary $\mathcal{A}$ that makes at most q_H (resp. $q_{H'}$) queries to H (resp. H'), there exists a OW-CPA adversary $\mathcal{B}$ s.t.*

$$\text{Adv}_{\text{KEM}}^{\text{ind-qcca}}(\mathcal{A}) \leq \delta + 2(2q_{H'} + q_H + q) \cdot 2^q \sqrt{((2q_{H'} + q))^q \cdot \text{Adv}_{\text{PKE}}^{\text{ow-cpa}}(\mathcal{B})}.$$

3.2 Hashing the Plaintext and Ciphertext

One can also wonder what is the leakage of the decapsulation oracle in the ROM, when the key is simply the hash of the seed and the plaintext. That is, we consider the simple PKE to KEM transform given in Fig. 7, which we call T_H . Note that this is the same transform as the one called $U^\perp$ in [19]. We now show that if q is small (logarithmic in the security parameter), then T_H holds a secure IND-qCCA scheme in the ROM, given that the underlying PKE is OW-CPA.

Theorem 3. *We consider a random oracle $H : \{0, 1\}^* \mapsto \{0, 1\}^n$. Let KEM be the KEM resulting from applying the T_H transform to a PKE PKE (which never queries H). Then, for any IND-qCCA adversary $\mathcal{A}$ that makes at most q_H queries to H , there exists a OW-CPA adversary $\mathcal{B}$ s.t.*

$$\text{Adv}_{\text{KEM}}^{\text{ind-qcca}}(\mathcal{A}) \leq q_H \cdot ((q_H + 1)(q_H + 2))^q \cdot \text{Adv}_{\text{PKE}}^{\text{ow-cpa}}(\mathcal{B}).$$

```

 $\mathcal{O}^i(\mathcal{L}_H, \text{ct})$ 
sort  $\mathcal{L}_H$  according to query order :
 $\mathcal{L}_H = ((\sigma_i, \text{ct}_i), K_i)_{i \in \{1, \dots, |\mathcal{L}_H|\}}$ 
 $\sigma' \leftarrow \text{dec}^P(\text{sk}, \text{ct})$ 
if  $\sigma' = \perp$  : return  $\perp_d$ 
for  $i \in \{1, \dots, |\mathcal{L}_H|\}$  :
    if  $\text{ct}_i = \text{ct}$  and  $\sigma' = \sigma_i$  :
        return  $i$ 
return  $\perp$ 

```

Fig. 8. $\mathcal{O}^i$ oracle for the proof of Theorem 3.

If PKE is deterministic, we get

$$\text{Adv}_{\text{KEM}}^{\text{ind-qcca}}(\mathcal{A}) \leq \delta + ((q_H + 1)(q_H + 2))^q \cdot \text{Adv}_{\text{PKE}}^{\text{ow-cpa}}(\mathcal{B}).$$

Proof. We start by defining an oracle $\mathcal{O}^i(\mathcal{L}_H, \text{ct})$ (see Fig. 8). This oracle returns the index i s.t. $((\sigma_i, \text{ct}_i), K_i) \in \mathcal{L}_H$ (we first sort $\mathcal{L}_H$ according to some fixed order) and $\text{ct}_i = \text{ct}$ and $\text{dec}^P(\text{sk}, \text{ct}_i) = \sigma_i$. If such a i does not exist and $\text{dec}^P(\text{sk}, \text{ct}_i) = \perp$ it returns $\perp_d$, otherwise it returns $\perp$.

Now we show how to simulate the IND-qCCA decapsulation oracle in the ROM, using $\mathcal{O}^i$ and $\mathcal{O}^{\text{PCO}}$ only. The original (resp. modified) oracles $\mathcal{O}^{\text{Dec}}$ and H (resp. $\mathcal{O}^{\text{Dec}'}$ and H') are on the left (resp. right) in Fig. 9. We now prove that any IND-qCCA adversary cannot distinguish between the real and modified oracles.

First, we show that the outputs of the ROs H and H' on any query (σ, ct) have the same distribution, given the adversary's view. We break this into four subcases:

- (σ, ct) was queried before to H (resp. H'): In this case, both H and H' return the value h returned on the previous similar query. Thus, we assume from now on that every RO query made by the adversary is unique.
- ct was never queried to the decapsulation oracle before: In this case, both H and H' return a random value h and store the query/response in $\mathcal{L}_H$.
- ct was queried to the decapsulation oracle before: In both cases (original and modified oracles) one can see that if the decryption of ct either fails or $\sigma' = \text{dec}^P(\text{sk}, \text{ct})$ is different from σ , then the output of the decapsulation oracle is independent of $H(\sigma, \text{ct})$ (and $H'(\sigma, \text{ct})$). In both cases, the ROs sample a fresh value (H' will do so because $\mathcal{O}^{\text{PCO}}(\sigma, \text{ct})$ will output 0 in this case, as $\sigma \neq \sigma'$ or the ciphertext is not valid). Now, if ct decrypts to σ , the original decapsulation oracle outputs $H(\sigma, \text{ct})$. In the modified game, the decapsulation oracle outputs a random K . Indeed, as we assume (σ, ct) was never queried to H , $\mathcal{O}^i(\mathcal{L}_H, \text{ct})$ outputs $\perp$. Then, the modified RO will output the same K , as $\mathcal{O}^{\text{PCO}}(\sigma, \text{ct})$ will verify. In both cases, the ROs output the same value as the decapsulation oracle.

Oracle $\mathcal{O}^{\text{Dec}}(\text{ct})$ if $\text{ct} = \text{ct}^*$: return $\perp$ if more than q queries : return $\perp$ $\sigma' \leftarrow \text{dec}^p(\text{sk}, \text{ct})$ if $\sigma' = \perp$: return $\perp$ return $H(\sigma', \text{ct})$ $H(\sigma, \text{ct})$ if $\exists h$ s.t. $((\sigma, \text{ct}), h) \in \mathcal{L}_H$: return h $h \leftarrow \mathbb{S}\{0, 1\}^n$ $\mathcal{L}_H \leftarrow \mathcal{L}_H \cup \{((\sigma, \text{ct}), h)\}$ return h	Oracle $\mathcal{O}^{\text{Dec}'}(\text{ct})$ if $\text{ct} = \text{ct}^*$: return $\perp$ if more than q queries : return $\perp$ if $\exists K$ s.t. $(\text{ct}, K) \in \mathcal{L}_K$: return K $i \leftarrow \mathcal{O}^i(\mathcal{L}_H, \text{ct})$ if $i = \perp_d$: return $\perp$ if $i \neq \perp$: $((\sigma_i, \text{ct}_i), K_i) \leftarrow \mathcal{L}_H[i]$ return K_i // return i-th valued returned by H' $K \leftarrow \mathbb{S}\{0, 1\}$ $\mathcal{L}_K \leftarrow \mathcal{L}_K \cup \{(\text{ct}, K)\}$ return K $H'(\sigma, \text{ct})$ if $\exists h$ s.t. $((\sigma, \text{ct}), h) \in \mathcal{L}_H$: return h if $\exists K$ s.t. $(\text{ct}, K) \in \mathcal{L}_K$: if $\mathcal{O}^{\text{PCO}}(\sigma, \text{ct})$: $\mathcal{L}_H \leftarrow \mathcal{L}_H \cup \{((\sigma, \text{ct}), K)\}$ return K $h \leftarrow \mathbb{S}\{0, 1\}^n$ $\mathcal{L}_H \leftarrow \mathcal{L}_H \cup \{((\sigma, \text{ct}), h)\}$ return h
---	---

Fig. 9. Original and modified oracles for the proof of Theorem 3.

We now show that the decapsulation oracles $\mathcal{O}^{\text{Dec}}$ and $\mathcal{O}^{\text{Dec}'}$ are indistinguishable. Let ct be the queried ciphertext and $\sigma = \text{dec}^p(\text{sk}, \text{ct})$.

- $\text{ct} = \text{ct}^*$: both oracles return $\perp$.
- $\sigma = \perp$: Both oracles return $\perp$, as $\mathcal{O}^i(\mathcal{L}_H, \text{ct})$ returns $\perp_d$.
- $H(\sigma, \text{ct})$ (resp. $H'(\sigma, \text{ct})$) was never queried. Both oracles return a random value if ct was never queried, or a consistent value if it was. It is straightforward to see this is the case in the original oracle. In the modified oracle, as $H'(\sigma, \text{ct})$ was never queried, we have $\mathcal{O}^i(\mathcal{L}_H, \text{ct})$ that returns $\perp$. Thus, the decapsulation oracle returns a random K if ct was not queried or a consistent K if it was.
- $H(\sigma, \text{ct})$ (resp. $H'(\sigma, \text{ct})$) was queried and it output K . Both oracles return K . In the modified decapsulation oracle, $\mathcal{O}^i(\mathcal{L}_H, \text{ct})$ will output a valid i s.t. $H'(\sigma_i, \text{ct}) = h_i$ and h_i is returned. Thus, the answer is consistent with the RO.

Now we can prove the theorem by game hopping as before. We define Γ^0 as the original IND-qCCA game.

$\mathcal{B}(\text{pk}, \text{ct}^*)$	Oracle $\mathcal{O}^{\text{Dec}''}(\text{ct})$
init $\mathcal{L}_H, \mathcal{L}_K \leftarrow \emptyset$	if $\text{ct} = \text{ct}^* : \text{return } \perp$
init $\mathcal{L}_q \leftarrow []$	if more than q queries : return $\perp$
$K^* \leftarrow \mathcal{K}$	if $\exists K$ s.t. $(\text{ct}, K) \in \mathcal{L}_K :$
run $\mathcal{A}^{H'', \mathcal{O}^{\text{Dec}''}}(\text{pk}, \text{ct}^*, K^*)$	return K
sample random query (σ', ct') made to H''	$i \leftarrow \$ \{1, \dots, q_H, \perp, \perp_d\}$
return σ'	if $i = \perp_d : \text{return } \perp$
	if $i \neq \perp :$
	$(\text{ct}_i, K_i) \leftarrow \mathcal{L}_H[i]$
	return K_i // return i -th valued returned by H''
$H''(\sigma, \text{ct})$	$K \leftarrow \$ \{0, 1\}$
$i_q \leftarrow$ query number	$\mathcal{L}_K \leftarrow \mathcal{L}_K \cup \{(\text{ct}, K)\}$
if $\exists h$ s.t. $((\sigma, \text{ct}), h) \in \mathcal{L}_H :$	$\mathcal{L}_q[\text{ct}] \leftarrow \$ \{0, \dots, q_H\}$
return h	return K
if $\exists K$ s.t. $(\text{ct}, K) \in \mathcal{L}_K :$	
if $\mathcal{L}_q[\text{ct}] = i_q :$	
$\mathcal{L}_H \leftarrow \mathcal{L}_H \cup \{((\sigma, \text{ct}), K)\}$	
return K	
$h \leftarrow \$ \{0, 1\}^n$	
$\mathcal{L}_H \leftarrow \mathcal{L}_H \cup \{((\sigma, \text{ct}), h)\}$	
return h	

Fig. 10. $\mathcal{B}$ adversary for the proof of Theorem 3.

$\underline{I^1}$: We modify the original IND-qCCA game into another game I^1 where the random/decapsulation oracles are the modified ones (i.e. H' and $\mathcal{O}^{\text{Dec}'}$) described above. As shown, both games are indistinguishable and thus

$$|\Pr[I^0 \Rightarrow 1] - \Pr[I^1 \Rightarrow 1]| = 0.$$

$\underline{I^2}$: We replace the challenge key by a random one, as in the previous proof. Then, similarly, the real key is indistinguishable from a random one unless $H(\sigma^*, \text{ct}^*)$ is queried. We define this event as **query** and

$$|\Pr[I^1 \Rightarrow 1] - \Pr[I^2 \Rightarrow 1]| \leq \Pr[\text{query}].$$

We can upper bound this probability by the advantage of a OW-CPA adversary $\mathcal{B}$ against PKE. That is, given a IND-qCCA adversary playing game I^2 , we build an adversary $\mathcal{B}$ as shown in Fig. 10. One can see that if $\mathcal{B}$ was simulating $\mathcal{A}$ with the H' and $\mathcal{O}^{\text{Dec}'}$ oracles (instead of its own oracles H'' and $\mathcal{O}^{\text{Dec}''}$), the simulation would be perfect as long as **query** did not occur. Then, whenever **query** would happen, $\mathcal{B}$ would recover σ^* with prob. $\frac{1}{q_H}$. Now $\mathcal{B}$ does not simulate the modified oracles perfectly but instead makes some guessing in its own oracles H'' and $\mathcal{O}^{\text{Dec}''}$:

- $\mathcal{O}^{\text{Dec}''}$: In line 5, i is picked at random instead of being the returned value of the $\mathcal{O}^i$ oracle. On each query the simulation is perfect with prob. $1/(q_H + 2)$ and overall with probability $\frac{1}{(q_H + 2)^q}$, as there are at most q queries to this oracle. In line 11, we associate a random index to each ct s.t. $(\text{ct}, *) \in \mathcal{L}_K$.

- H'' : In line 5, when $(\text{ct}, *) \in \mathcal{L}_K$, instead of querying the plaintext-checking oracle we check whether the corresponding sampled index $\mathcal{L}_q[\text{ct}]$ is equal to the query number. If it is, we reply with K s.t. $(\text{ct}, K) \in \mathcal{L}_K$ otherwise we proceed as before (i.e. as in H'). Let's assume w.l.o.g that each query to H'' is unique. For each ct s.t. $(\text{ct}, *) \in \mathcal{L}_K$, there can be at most one query (σ, ct) s.t. $\mathcal{O}^{\text{PCO}}(\sigma, \text{ct})$ returns 1 (it is when σ is the decryption of ct). Here, $\mathcal{B}$ guesses beforehand which query it is (or if no such query will be made) and gets the correct answer with prob. $\frac{1}{q_H+1}$. Note that $\mathcal{B}$ needs to make one guess per query to $\mathcal{O}^{\text{Dec}''}$ (not per query to H''). Overall, the probability H'' simulates correctly H' is $\frac{1}{(q_H+1)^q}$.

From this we can deduce that $\mathcal{B}$ correctly simulates Γ^2 with probability $\frac{1}{((q_H+1)(q_H+2))^q}$ and wins the OW-CPA game with prob. at least $\frac{1}{q_H} \cdot \Pr[\text{query}]$. Hence,

$$|\Pr[\Gamma^1 \Rightarrow 1] - \Pr[\Gamma^2 \Rightarrow 1]| \leq \Pr[\text{query}] \leq q_H \cdot ((q_H+1)(q_H+2))^q \cdot \text{Adv}_{\text{PKE}}^{\text{ow-cpa}}(\mathcal{B}).$$

Note that when PKE is deterministic, in order to recover σ^* , $\mathcal{B}$ can check which σ' queried is s.t. $\text{enc}(\text{pk}^*, \sigma') = \text{ct}^*$ instead of guessing. This works as long as the challenge seed σ^* and queried seeds are correct. If that is not the case, one can build an adversary that wins the correctness game defined in Fig. 1. Note that this adversary knows which will be the correct seed as it is given sk and the PKE is deterministic. As the correctness advantage is upper bounded by δ , we obtain that for deterministic PKEs the last inequality becomes

$$|\Pr[\Gamma^1 \Rightarrow 1] - \Pr[\Gamma^2 \Rightarrow 1]| \leq \Pr[\text{query}] \leq \delta + ((q_H+1)(q_H+2))^q \cdot \text{Adv}_{\text{PKE}}^{\text{ow-cpa}}(\mathcal{B}).$$

Finally, in game Γ^2 , the challenge key is always random and thus $\Pr[\Gamma^2 \Rightarrow 1] = \frac{1}{2}$. Collecting the probabilities holds the result. $\square$

4 CPA-security Is Sufficient for TLS 1.3 in the ROM

We show in this section that a CPA-secure KEM is sufficient for the handshake in TLS 1.3 to be secure in the ROM. The security bound is very loose, but this still solves an interesting open problem. TLS 1.3 only supports DH key-exchange but it can be trivially modified to support KEMs as done in several PQ variants of TLS (e.g. [1, 7]). That is, the client runs $(\text{sk}, \text{pk}) \leftarrow \text{gen}$ and sends pk as its share (instead of g^x). Then, the server runs $K, \text{ct} \leftarrow \text{encaps}$ and sends ct as its secret share (instead of g^y). Finally, the client runs $K \leftarrow \text{decaps}(\text{sk}, \text{ct})$ and the shared secret is set to K . By abuse of language, we refer to this modified protocol as TLS 1.3 in what follows. An overview of this modified handshake is given in Fig. 14.

IND-1CCA-MAC_{KEM}($\mathcal{A}$)

```

 $b \leftarrow \$ \{0, 1\}$ 
 $(pk, sk) \leftarrow \$ \text{gen}()$ 
 $ct^*, K^* \leftarrow \$ \text{encaps}(pk)$ 
 $n^* \leftarrow \$ \{0, 1\}^n$ 
 $HS^* \leftarrow G(K^*)$ 
 $CHTS_0 \leftarrow H_1(HS^*, H_T(ct^*, n^*))$ 
 $SHTS_0 \leftarrow H_2(HS^*, H_T(ct^*, n^*))$ 
 $dHS_0 \leftarrow H_3(HS^*)$ 
 $(CHTS_1, SHTS_1, dHS_1) \leftarrow \$ \{0, 1\}^{3n}$ 
 $b' \leftarrow \mathcal{A}^{\mathcal{O}^{\text{Dec}}, \mathcal{O}^{\text{MAC}}}_{\text{MAC}}(pk, ct^*, n^*, (CHTS_b, SHTS_b, dHS_b))$ 
return  $1_{b'=b}$ 

```

Oracle $\mathcal{O}^{\text{Dec}}((ct, n))$

```

if more than 1 query : return  $\perp$ 
if  $(ct, n) = (ct^*, n^*)$  : return  $\perp$ 
 $K' \leftarrow \text{decaps}(sk, ct)$ 
if  $K' = \perp$  : return  $\perp$ 
 $HS' \leftarrow G(K')$ 
 $CHTS \leftarrow H_1(HS', H_T(ct, n))$ 
 $SHTS \leftarrow H_2(HS', H_T(ct, n))$ 
 $tk_c \leftarrow H_D(CHTS); tk_s \leftarrow H_D(SHTS)$ 
return  $(tk_c, tk_s)$ 

```

Oracle $\mathcal{O}^{\text{Dec}}_{\text{MAC}}(ct, n, \text{tag}, \text{txt})$

```

if more than 1 query : return  $\perp$ 
if  $(ct, n) = (ct^*, n^*)$  : return  $\perp$ 
 $K' \leftarrow \text{decaps}(sk, ct)$ 
 $HS' \leftarrow G(K'); SHTS \leftarrow H_2(HS', H_T(ct, n))$ 
 $fk_S \leftarrow H_4(SHTS)$ 
if  $\text{MAC.Vrf}(fk_S, \text{txt}, \text{tag}) = \text{true}$  :
    return  $HS'$ 
return  $\perp$ 

```

Fig. 11. IND-1CCA-MAC game.**4.1 IND-1CCA-MAC**

In order to show that a CPA-secure KEM is sufficient for TLS 1.3 to be secure, we first introduce an intermediary notion of security for KEMs, called IND-1CCA-MAC. This security definition has no application and will serve only as a useful intermediary building block for the proof.

Definition 6 (IND-1CCA-MAC). *We consider the games defined in Fig. 11. Let $\mathcal{K}$ be the key space, G, H_1, H_2, H_3, H_4 and H_D be key-derivation functions with images in $\{0, 1\}^n$, H_T be a hash function with images in $\{0, 1\}^n$, and MAC a MAC scheme. A KEM scheme $\text{KEM} = (\text{gen}, \text{encaps}, \text{decaps})$ is IND-1CCA-MAC if for any ppt adversary $\mathcal{A}$ we have*

$$\text{Adv}_{\text{KEM}}^{\text{ind-1cca-mac}}(\mathcal{A}) := \left| \Pr[\text{IND-1CCA-MAC}_{\text{KEM}}(\mathcal{A}) \Rightarrow 1] - \frac{1}{2} \right| = \text{negl}(\lambda)$$

where $\Pr[\text{IND-1CCA-MAC}_{\text{KEM}}^b(\mathcal{A}) \Rightarrow 1]$ is the probability that $\mathcal{A}$ wins the IND-1CCA-MAC_{KEM}^b($\mathcal{A}$) game defined in Fig. 11.

In this game, the adversary receives a challenge ciphertext encapsulating a key K , a nonce n^* , and either three secrets $(CHTS_b, SHTS_b, dHS_b)$ derived from K through a key schedule, or three random secrets. Jumping ahead, these three

values are computed (nearly) in the same as way as their identically named counterparts in the modified TLS 1.3 protocol. The adversary has also access to two oracles that it can query *at most once*. The first is simply a decapsulation oracle that applies a key schedule (similar to TLS's) on the decapsulated key and returns two secrets tk_c and tk_s . The second oracle takes a ciphertext (which must be different than the challenge ciphertext), a tag, and some data. Then, the ciphertext is decrypted to recover a secret HS' that is passed through a key schedule to get a MAC key fk_s . Finally, the oracle checks whether tag is a valid MAC on the data with the key fk_s . If this is the case it returns HS' , otherwise it returns an error $\perp$. Informally, this last oracle outputs the root secret HS if the adversary can forge a valid tag corresponding to the tuple (ct, n) . In the TLS proof, this will be used to argue that if a participant can send a valid tag, it should know the root secret HS .

4.2 OW-CPA Implies IND-1CCA-MAC

First, we briefly define the notion of MAC unforgeability we will need.

Definition 7 (MAC EUF-0T). *Let $\text{MAC} = (\text{MAC.Vrf}, \text{MAC.Tag})$ be a message authentication code scheme (MAC). We say MAC is EUF-0T if for any ppt adversary $\mathcal{A}$,*

$$\text{Adv}_{\text{MAC}}^{\text{euf-0t}}(\mathcal{A}) := \Pr[\text{MAC.Vrf}(K, M, T) = 1 : (M, T) \leftarrow_{\$} \mathcal{A}; K \leftarrow_{\$} \mathcal{K}]$$

is negligible in the security parameter, where the probability is taken over the sampling of the key and the randomness of the adversary.

We now prove that any OW-CPA KEM is also IND-1CCA-MAC secure in the ROM if the MAC used is EUF-0T secure. More precisely, the KDFs G, H_1, H_2, H_3, H_4 and H_D , and the hash function H_T in the IND-1CCA-MAC games are assumed to be ROs.

Theorem 4. *Let $\text{KEM} = (\text{gen}, \text{encaps}, \text{decaps})$ be a KEM. Let the KDFs and the hash function in the IND-1CCA-MAC game be modelled as random oracles. Then, for any ppt adversary $\mathcal{A}$ making at most $q_G, q_{H_1}, q_{H_2}, q_{H_3}, q_{H_4}, q_{H_D}, q_{H_T}$ queries to $G, H_1, H_2, H_3, H_4, H_D, H_T$ respectively, there exists a OW-CPA adversary $\mathcal{B}$ s.t.*

$$\begin{aligned} \text{Adv}_{\text{KEM}}^{\text{ind-1cca-mac}}(\mathcal{A}) &\leq \text{Adv}_{\text{MAC}}^{\text{euf-0t}}(\mathcal{B}) + \frac{3q_{H_1} + 4q_{H_2} + q_{H_3} + q_{H_4} + q_{H_D} + 1}{2^n} \\ &\quad + \frac{(q_{H_T} + 4)^2}{2^n} + q_G(q_{H_1} + 2)^2(q_{H_2} + 2)^3 \cdot \text{Adv}_{\text{KEM}}^{\text{ow-cpa}}(\mathcal{C}), \end{aligned}$$

where $\mathcal{B}$ has approximately the same running time as $\mathcal{A}$.

Proof. The first step of the proof is very similar to the proof of Theorem 3. Indeed, one can see that the decapsulation oracle outputs secrets that are computed as (a function of) $H_i(\text{HS}, H_T(\text{ct}, n))$, where H_i and H_T are ROs. Note that

$\Gamma_{\text{KEM}}^{0-6}(\mathcal{A})$	Oracle $\mathcal{O}_{\text{MAC}}^{\text{Dec}}(\text{ct}, n, \text{tag}, \text{txt})$
$b \leftarrow \$ \{0, 1\}$ $(\text{pk}, \text{sk}) \leftarrow \$ \text{gen}()$ $\text{ct}^*, K^* \leftarrow \$ \text{encaps}(\text{pk})$ $n^* \leftarrow \$ \{0, 1\}^n$ $\text{HS}^* \leftarrow G(K^*)$ $\text{CHTS}_0 \leftarrow H_1(\text{HS}^*, H_T(\text{ct}^*, n^*))$ $\text{SHTS}_0 \leftarrow H_2(\text{HS}^*, H_T(\text{ct}^*, n^*))$ $\text{dHS}_0 \leftarrow H_3(\text{HS}^*)$ $(\text{CHTS}_1, \text{SHTS}_1, \text{dHS}_1) \leftarrow \$ \{0, 1\}^{3n}$ $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{MAC}}^{\text{Dec}}, \mathcal{O}_{\text{MAC}}^{\text{Dec}}, H_1, H_2}(\text{pk}, \text{ct}^*, n^*,$ $\quad (\text{CHTS}_b, \text{SHTS}_b, \text{dHS}_b)) // \Gamma^0 - \Gamma^3$ $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{MAC}}^{\text{Dec}'}, \mathcal{O}_{\text{MAC}}^{\text{Dec}}, H_1', H_2'}(\text{pk}, \text{ct}^*, n^*,$ $\quad (\text{CHTS}_b, \text{SHTS}_b, \text{dHS}_b)) // \Gamma^4 -$ if collision on H_T : abort // $\Gamma^1 -$ if $\mathcal{A}$ queries $H_i(\text{HS}^*, H_T(\text{ct}^*, n^*))$, $i \in [2]$ or $H_3(\text{HS}^*)$: $\quad$ abort // Γ^6 if $\mathcal{A}$ did not query $G(K^*)$: abort // Γ^5 return $1_{b'=b}$	if more than 1 query : return $\perp$ if $(\text{ct}, n) = (\text{ct}^*, n^*)$: return $\perp$ $K' \leftarrow \text{decaps}(\text{sk}, \text{ct})$ $\text{HS}' \leftarrow G(K')$; $\text{SHTS} \leftarrow H_2(\text{HS}', H_T(\text{ct}, n))$ $\text{fk}_S \leftarrow H_4(\text{SHTS})$ if $\text{SHTS} = \text{SHTS}_b$: // $\Gamma^2 -$ $\quad$ abort // $\Gamma^2 -$ if $\text{MAC.Vrf}(\text{fk}_S, \text{txt}, \text{tag}) = \text{true}$: $\quad$ if $\mathcal{A}$ did not query $H_4(\text{SHTS})$: // $\Gamma^2 -$ $\quad \quad$ abort // $\Gamma^2 -$ $\quad$ if $\mathcal{A}$ did not query $H_2(\text{HS}', H_T(\text{ct}, n))$: // $\Gamma^3 -$ $\quad \quad$ abort // $\Gamma^3 -$ $\quad$ return HS' return $\perp$ $H_j(\text{HS}, y)$, $j \in \{1, 2\}$ if $\nexists (\text{ct}, n)$ s.t. $((\text{ct}, n), y) \in \mathcal{L}_{H_T}$: // $\Gamma^1 -$ $\quad h \leftarrow \$ \{0, 1\}^n$; return h // $\Gamma^1 -$ usual lazy sampling

Fig. 12. Games for the proof of Theorem 4. The adversary has access to all the other ROs G, H_3, H_4 and H_D , even if it is not explicited in the games. H'_1, H'_2 and $\mathcal{O}_{\text{MAC}}^{\text{Dec}'}$ are defined in Fig. 13.

the only difference is that H_T is applied on (ct, n) . However, as H_T is a RO, this difference will not matter much in the proof. Hence, as in Theorem 3, one can program the ROs s.t. the decapsulation oracle $\mathcal{O}^{\text{Dec}}$ can be simulated without the secret key. In a second step, we show that the adversary can also simulate the $\mathcal{O}_{\text{MAC}}^{\text{Dec}}$ oracle with good probability. More precisely, let HS be the secret corresponding to the submitted ciphertext ct . Then, either $H_2(\text{HS}, H_T(\text{ct}, n))$ has been queried by the adversary or it is very unlikely that $\mathcal{A}$ knows the MAC key fk_S . In the first case we can recover HS from the list of queries, and in the second we can return $\perp$ as most likely the MAC verification will fail.

We proceed with a sequence of games, which are given in detail in Fig. 12.

Γ^0 : This is the original IND-1CCA-MAC game. From now on, we assume w.l.o.g. that each query to ROs are unique (i.e. they never repeat).

Γ^1 : We modify the previous game as follows. First, we abort if a collision on H_T occurs in the game. As there are at most $q_{H_T} + 4$ queries to H_T in the game, a collision occurs with prob. less than $\frac{(q_{H_T} + 4)^2}{2^n}$. Then, on adversary's queries $H_j(\text{HS}, y)$, $j \in \{1, 2\}$, if $H_T(\text{ct}, n) = y$ was never queried by $\mathcal{A}$ for some (ct, n) , we mark y as *unpaired* and return a random value. The only way it differs from the previous game, is if a query $H_j(\text{HS}, y)$ for an *unpaired* y is performed by the game (i.e. not by the adversary), either before or after y was marked as *unpaired*. Now, the $\mathcal{A}$ does not get any information about values $H_T(\text{ct}, n)$ from the game

(or oracles), except a few values $H_j(\text{HS}, H_T(\text{ct}, n))$ (or values that depends on these), for some HS . Note that these values completely “hide” the result of the H_T query, as H_j is a RO. Hence, the best strategy for $\mathcal{A}$ to query $H_j(\text{HS}, y)$ s.t. y is *unpaired* but is queried by the game at some point, is to try random values for y . As the game makes at most 2 queries to H_1 (one in the challenge part and one in the decapsulation oracle) and 3 queries to H_2 (one in the challenge part and one in each oracle), the probability that a random unpaired y is s.t. y was the result of a H_T query by the game at some point, is at most $\frac{2}{2^n}$ for a H_1 call, and $\frac{3}{2^n}$ for a H_2 call. Overall, we have

$$|\Pr[\Gamma^0 \Rightarrow 1] - \Pr[\Gamma^1 \Rightarrow 1]| \leq \frac{2q_{H_1} + 3q_{H_2}}{2^n}.$$

We note that this step ensures that on a query $H_j(\text{HS}, y)$, one can recover a unique tuple (ct, n) s.t. $H_T(\text{ct}, n) = y$, or a random value is returned.

Γ^2 : We modify the original game s.t. we abort whenever the MAC verification succeeds on the query $\mathcal{O}_{\text{MAC}}^{\text{Dec}}(\text{ct}, n, \text{tag}, \text{txt})$ but $\text{fk}_S := H_4(\text{SHTS})$ was never queried, where $\text{SHTS} := H_2(G(K), H_T(\text{ct}, n))$ and $K := \text{decaps}(\text{sk}, \text{ct})$. If that is the case, it means the MAC key $\text{fk}_S := H_4(\text{SHTS})$ is indistinguishable from a random value for $\mathcal{A}$, but it managed to forge a valid tag. Thus, one can build an adversary $\mathcal{B}$ that breaks MAC unforgeability. More formally, $\mathcal{B}$ samples a pair of keys $(\text{sk}, \text{pk}) \leftarrow \text{gen}$, generates a valid input for $\mathcal{A}$ and simulates the decryption oracle with the secret key. Then, when $\mathcal{A}$ submits $(\text{ct}, n, \text{tag}, \text{txt})$ to $\mathcal{O}_{\text{MAC}}^{\text{Dec}}$, $\mathcal{B}$ outputs (txt, tag) as a forgery. We also abort if the value SHTS computed in the oracle is s.t. $\text{SHTS} = \text{SHTS}_b$. As there are no collision on H_T and $(\text{ct}, n) \neq (\text{ct}^*, n^*)$, this happens with probability at most $\frac{1}{2^n}$. Then, we have

$$|\Pr[\Gamma^1 \Rightarrow 1] - \Pr[\Gamma^2 \Rightarrow 1]| \leq \text{Adv}_{\text{MAC}}^{\text{uf-0t}}(\mathcal{B}) + \frac{1}{2^n}.$$

Γ^3 : We abort whenever the MAC verification succeeds on the query $\mathcal{O}_{\text{MAC}}^{\text{Dec}}(\text{ct}, n, \text{tag}, \text{txt})$ but $H_2(G(K), H_T(\text{ct}, n))$ was never queried, where $K := \text{decaps}(\text{sk}, \text{ct})$. By the previous game, it means that the adversary queried $\text{SHTS} := H_2(G(K), H_T(\text{ct}, n))$ to H_4 without having queried $H_2(G(K), H_T(\text{ct}, n))$ beforehand. If we analyse what information $\mathcal{A}$ has about $\text{SHTS} \neq \text{SHTS}_b$ if it did not query $H_2(G(K), H_T(\text{ct}, n))$, we see that the only potential “leakage” is from a decapsulation query that returns $\text{tk}_s := H_D(\text{SHTS})$, where H_D is a RO perfectly hiding SHTS .

Thus, the best strategy for $\mathcal{A}$ to find SHTS without querying H_2 is to query random values $x \in \{0, 1\}^n$ to H_D or H_4 until it finds x s.t. $H_D(x) = \text{tk}_s$ or $H_4(x) = \text{fk}_S$. This happens with probability at most $\frac{q_{H_D} + q_{H_4}}{2^n}$. Hence, we have

$$|\Pr[\Gamma^2 \Rightarrow 1] - \Pr[\Gamma^3 \Rightarrow 1]| \leq \frac{q_{H_D} + q_{H_4}}{2^n}.$$

Oracle $\mathcal{O}^{\text{Dec}'}(\text{ct}, n)$	$H'_j(\text{HS}, y), j \in \{1, 2\}$
if $(\text{ct}, n) = (\text{ct}^*, n^*)$: return $\perp$ if more than 1 query : return $\perp$ $q_1 \leftarrow \$\{0, \dots, q_{H_1}\}$ $q_2 \leftarrow \$\{0, \dots, q_{H_2}\}$ $i \leftarrow \mathcal{O}^i(\mathcal{L}_{H_1}, \text{ct}, n)$ if $i = \perp_d$: return $\perp$ if $i \neq \perp$: // get i -th valued returned by H_1 $((\text{HS}_i, \text{ct}_i, n_i), h_i) \leftarrow \mathcal{L}_{H_1}[i]$ $\text{CHTS} \leftarrow h_i$ else : $\text{CHTS} \leftarrow \$\{0, 1\}$ $\mathcal{L}_K^1 \leftarrow (\text{ct}, n, \text{CHTS})$ $i \leftarrow \mathcal{O}^i(\mathcal{L}_{H_2}, \text{ct}, n)$ if $i \neq \perp$: // get i -th valued returned by H_2 $((\text{HS}_i, \text{ct}_i, n_i), h_i) \leftarrow \mathcal{L}_{H_2}[i]$ $\text{SHTS} \leftarrow h_i$ else : $\text{SHTS} \leftarrow \$\{0, 1\}$ $\mathcal{L}_K^2 \leftarrow (\text{ct}, n, \text{SHTS})$ return $(H_D(\text{CHTS}), H_D(\text{SHTS}))$	if $\nexists (\text{ct}, n)$ s.t. $((\text{ct}, n), y) \in \mathcal{L}_{H_T}$: $h \leftarrow \$\{0, 1\}^n$; return h set (ct, n) s.t. $((\text{ct}, n), y) \in \mathcal{L}_{H_T}$ if $\mathcal{L}_K^j = (\text{ct}, n, h)$ for some h : if $\text{HS} = G(\text{decaps}(\text{sk}, \text{ct}))$: $\mathcal{L}_{H_j} \leftarrow \mathcal{L}_{H_j} \cup \{((\text{HS}, \text{ct}, n), h)\}$ return h $h \leftarrow \$\{0, 1\}^n$ $\mathcal{L}_{H_j} \leftarrow \mathcal{L}_{H_j} \cup \{((\text{HS}, \text{ct}, n), h)\}$ return h
	$\mathcal{O}_G^i(\mathcal{L}, n, \text{ct})$ sort $\mathcal{L}$ according to query order : $\mathcal{L} = ((\text{HS}_i, \text{ct}_i, n_i), h_i)_{i \in \{1, \dots, \mathcal{L}_H \}}$ $K' \leftarrow \text{decaps}(\text{sk}, \text{ct})$ if $K' = \perp$: return $\perp_d$ $\text{HS}' \leftarrow G(K')$ for $i \in \{1, \dots, \mathcal{L} \}$: if $(\text{ct}_i, n_i) = (\text{ct}, n)$ and $\text{HS}' = \text{HS}_i$: return i return $\perp$

Fig. 13. Simulation of decapsulation and random oracles with sub-oracle $\mathcal{O}_G^i$ for the proof of Theorem 4. Note that as we assume that each query to H_j is unique, H'_j does not check whether a query was previously made.

$\underline{I}^4$: We program both ROs H_1 and H_2 s.t. we can perfectly simulate the decapsulation oracle with an oracle $\mathcal{O}_G^i$. This follows exactly the idea of the proof of Theorem 3. First, we introduce an oracle $\mathcal{O}_G^i$ in Fig. 13 that takes a list of RO queries, a nonce n and a ciphertext ct , checks whether $(G(K), H_T(\text{ct}, n))$ (where K is the key encapsulated in ct) was ever queried and if that is the case, the index of the corresponding query. This is exactly the same as the oracle $\mathcal{O}^i$ in the proof of Theorem 3, except we query the decapsulated K to the RO G and there is the additional nonce. Then, we can program the ROs H_j , $j \in \{1, 2\}$ and simulate the (1-time) decapsulation oracle as shown in Fig. 13.

The simulation works nearly as in the proof of Theorem 3. Let ct be the unique decapsulation query, $K := \text{decaps}(\text{sk}, \text{ct})$ and $\text{HS} := G(K)$. For $j \in [2]$, the simulated decapsulation oracle checks whether $(G(K), H_T(\text{ct}, n))$ was already queried to H_j using $\mathcal{O}_G^i$, if that is the case it recovers the corresponding value, otherwise it means $H_j(\text{HS}, H_T(\text{ct}, n))$ was never queried by the adversary *nor* the challenger, as $(\text{ct}, n) \neq (\text{ct}^*, n^*)$. Thus it samples the hash value at random, queries it to H_D and returns it to the adversary.

The simulation of H_j is such that it is consistent with the values returned by the simulated decapsulation oracle. First, if $H_j(\text{HS}, y)$ is queried s.t. y is *unpaired*, we can simply return a random value, this is consistent with the game. Then, if y is not *unpaired*, one can recover the unique (as there are no collision) tuple (ct, n) s.t. $y = H_T(\text{ct}, n)$. We consider from now on only queries with y s.t. $H_T(\text{ct}, n) = y$ for some (ct, n) . On a query $H_j(\text{HS}, H_T(\text{ct}, n))$, if (ct, n) was already queried to the decapsulation oracle, then $h := H_j(\text{HS}, \text{ct}, n)$ was set by $\mathcal{O}^{\text{Dec}'}$ iff $\text{HS} = G(K)$, where $K := \text{decaps}(\text{sk}, \text{ct})$. Hence, we return the same K if $G(\text{decaps}(\text{sk}, \text{ct})) = \text{HS}$. Otherwise we sample a random value and return it. Note that this is the only place where the secret key sk is used anymore (except implicitly in the $\mathcal{O}_G^i$ oracle). The simulation is perfect and therefore we have

$$|\Pr[\Gamma^3 \Rightarrow 1] - \Pr[\Gamma^4 \Rightarrow 1]| = 0.$$

Γ^5 : In game Γ^5 , we abort whenever the adversary did not query $G(K^*)$ (which is equal to HS^*) but it queried $H_1(\text{HS}^*, H_T(\text{ct}^*, n^*))$, $H_2(\text{HS}^*, H_T(\text{ct}^*, n^*))$ or $H_3(\text{HS}^*)$. Note that the (modified) decryption oracle never queries $H_1(\text{HS}^*, H_T(\text{ct}^*, n^*))$, $H_2(\text{HS}^*, H_T(\text{ct}^*, n^*))$ or $H_3(\text{HS}^*)$. In addition, the challenge values given to $\mathcal{A}$ are either perfectly random or completely hide HS^* . Thus, the probability that $\mathcal{A}$ queries HS^* to H_1, H_2 or H_3 is upper bounded by $\frac{q_{H_1} + q_{H_2} + q_{H_3}}{2^n}$ and hence we have

$$|\Pr[\Gamma^4 \Rightarrow 1] - \Pr[\Gamma^5 \Rightarrow 1]| \leq \frac{q_{H_1} + q_{H_2} + q_{H_3}}{2^n}.$$

Γ^6 : Finally, in game Γ^6 we abort whenever $H_1(\text{HS}^*, H_T(\text{ct}^*, n^*))$, $H_2(\text{HS}^*, H_T(\text{ct}^*, n^*))$ or $H_3(\text{HS}^*)$ is queried by the adversary. Let *query* be this event. By the previous game, it means that K^* was queried to G before *query* happens. Finally, as in the previous proofs, we can upper bound $\Pr[\text{query}]$ by the advantage of a OW-CPA adversary times a constant. The challenge keys $(\text{CHTS}_b, \text{SHTS}_b, \text{dHS}_b)$ are sampled at random in the reduction, as long *query* does not happen both the real and random cases are perfectly indistinguishable. The only challenge for such a OW-CPA adversary $\mathcal{C}$ is to simulate the oracles without having access to the secret key. It proceeds as follows (see the full version of the paper [20] for a complete and detailed presentation of $\mathcal{C}$).

- $\mathcal{O}_{\text{MAC}}^{\text{Dec}''}$: This oracle returns something else than $\perp$ iff $(\text{HS}, H_T(\text{ct}, n))$ was queried to H_2 , where $\text{HS} := G(K)$ and $K := \text{decaps}(\text{sk}, \text{ct})$. Hence, one can simply pick a random value $r \leftarrow \{0, \dots, q_{H_2}\}$ and guess whether $\mathcal{O}_{\text{MAC}}^{\text{Dec}}(\text{ct})$ fails (if $r = 0$) or succeeds and HS is in the r -th query made to H_2 . In the latter case, one can recover HS in the r -th query and return it. Overall the simulation works with probability $\frac{1}{q_{H_2} + 1}$.
- $\mathcal{O}^{\text{Dec}''}$: In this oracle, the secret-key is used only in the $\mathcal{O}_G^i$ sub-oracle. A reply of $\mathcal{O}_G^i$ is in the set $\{\perp, \perp_d, 1, \dots, q_{H_j}\}$ for $j \in [2]$. Thus, one can guess the correct reply by sampling a random value in that set, which gives a success probability of $\frac{1}{(q_{H_j} + 2)}$. Overall, there are at most 2 calls to $\mathcal{O}_G^i$ (one for $j = 1$

and $j = 2$) and therefore the probability that the simulation is successful is $\frac{1}{(q_{H_1}+2)(q_{H_2}+2)}$.

- H_j'' , $j \in [2]$: The only time the secret key is used is when there is a query $(\text{HS}, H_T(\text{ct}, n))$ s.t. (ct, n) was already queried to $\mathcal{O}^{\text{Dec}'}$ (i.e. $\mathcal{L}_K^j = (\text{ct}, n, h)$ for some h). In this case h is returned iff $G(\text{decaps}(\text{sk}, \text{ct})) = \text{HS}$ (let's call this Condition (1)). Recalling that queries to H_j never repeat by assumption, there will be at most one query $H_j''(\text{HS}, H_T(\text{ct}, n))$ s.t. (ct, n) was queried to the decapsulation oracle and Condition (1) is fulfilled. Hence, one can simulate H_j by sampling an index $q_j \in \{0, \dots, q_{H_j}\}$ and returning h (if it exists) in the q_j -th query or never in case $q_j = 0$. This successfully simulates H_j with prob. $\frac{1}{(q_{H_j}+1)}$. Overall, the probability that both H_1 and H_2 are simulated correctly is $\frac{1}{(q_{H_1}+1)(q_{H_2}+1)}$.

The other ROs can be simulated perfectly by $\mathcal{C}$ using lazy sampling. Collecting the probabilities holds that $\mathcal{C}$ simulates perfectly $\mathcal{A}$'s view in game Γ^6 (as long as query does not occur) with probability

$$p = \frac{1}{(q_{H_2}+1)^2(q_{H_1}+2)(q_{H_2}+2)(q_{H_1}+1)}.$$

Then if query happens, K^* will be in the list of queries made by $\mathcal{A}$ to G . The adversary can guess which one it is and succeeds with probability $\frac{1}{q_G}$. Hence, we have

$$|\Pr[\Gamma^5 \Rightarrow 1] - \Pr[\Gamma^6 \Rightarrow 1]| \leq \Pr[\text{query}] \leq q_G(q_{H_1}+2)^2(q_{H_2}+2)^3 \cdot \text{Adv}_{\text{KEM}}^{\text{ow-cpa}}(\mathcal{C}).$$

Finally, in game Γ^6 , as $H_1(\text{HS}^*, \text{ct}^*, n^*)$, $H_2(\text{HS}^*, \text{ct}^*, n^*)$ or $H_3(\text{HS}^*)$ cannot be queried anymore, the challenge keys are perfectly indistinguishable from random for the adversary. Hence,

$$\Pr[\Gamma^6 \Rightarrow 1] = \frac{1}{2}.$$

Collecting the probabilities holds the result. $\square$

4.3 Security of TLS 1.3 with IND-1CCA-MAC KEM

We can now use the (slightly modified) notion IND-1CCA-MAC KEM to prove the security of the TLS 1.3 handshake in the multi-stage security model of Günther [17]. We provide a brief reminder of the notion of MultiStage security in the full version of the paper [20]. The security of (the original) TLS 1.3 handshake was proven by Dowling et al. [12] and we refer the reader to their work for a complete analysis of the handshake. We will simply show that IND-1CCA-MAC KEMs, thus OW-CPA KEMs (if the MAC is secure), can be used in place of the original snPRF-ODH assumption for DH key-exchange.

First, we show the relevant part of the (full 1-RTT) handshake of TLS 1.3 in Fig. 14. One can see that the key schedule is nearly identical to the ones

used in the IND-1CCA-MAC game. Note that several simplifications have been made and several steps irrelevant to our proofs are missing. In particular, we do not see the derivation of the final keys, which all depend on the secret dHS . As we will show, the intermediary secrets $(\text{CHTS}, \text{SHTS}, \text{dHS})$ are secure (i.e. indistinguishable from random for a Multi-Stage adversary), thus all subsequent keys will be secure as well, assuming the KDFs are secure. Finally, we write $\text{HKDF.Exp}_i(\text{HS}, T_2)$ for $\text{HKDF.Exp}(\text{HS}, \text{label}_i, T_2)$, where label_i is some string. As we consider the KDFs HKDF.Ext and HKDF.Exp to be ROs, this denotes the fact that the label implements oracle separation.

The security of the modified 1-RTT TLS 1.3 handshake is stated in the following theorem.

Theorem 5. *Let HKDF.Ext , HKDF.TK and HKDF.Exp_j , $j \in \{0, 4, 5, 6\}$ (the KDFs in TLS 1.3) be random oracles. Let Hash (the hash function used to compute the hashed transcripts T_i) be a RO, and Sig the signature scheme used for server authentication (not shown in Fig. 14). For any Multi-Stage ppt adversary $\mathcal{A}$ there exist ppt adversaries $\{\mathcal{B}_i\}_{i \in [6]}$ s.t.*

$$\begin{aligned} \text{Adv}_{\text{TLS1.3-1RTT}}^{\text{multi-stage}}(\mathcal{A}) &\leq 6t_s \left(\text{Adv}_H^{\text{coll}}(\mathcal{B}_1) + t_u \text{Adv}_{\text{Sig}}^{\text{euf-cma}}(\mathcal{B}_2) \right. \\ &\quad + t_s \left(\text{Adv}_{\text{KEM}}^{\text{ind-1cca-mac}}(\mathcal{B}_3) + 2 \cdot \text{Adv}_{\text{HKDF.Exp}}^{\text{prf}}(\mathcal{B}_4) \right. \\ &\quad \left. \left. + \text{Adv}_{\text{HKDF.Ext}}^{\text{prf}}(\mathcal{B}_5) + \text{Adv}_{\text{HKDF.Exp}}^{\text{prf}}(\mathcal{B}_6) \right) \right). \end{aligned}$$

where t_s (resp. t_u) is the maximal number of sessions (resp. users). Note that for the sake of the comparison with the original bound, we keep several PRF advantages and the collision advantage in the bound, even though they could be replaced by negligible terms, as the KDFs and Hash are ROs.

Proof sketch. Due to space constraint, we defer the proof to the full version of the paper [20]. However, as hinted above, the idea of the proof is simply to replace the snPRF-ODH step of the original proof by using our IND-1CCA-MAC. Note that while the snPRF-ODH assumption is used to replace the root secret HS by a random one, we will be able to replace the values $(\text{CHTS}, \text{SHTS}, \text{dHS})$ by random ones in one step, due to the structure of the IND-1CCA-MAC definition. From a high-level point of view, the proof goes through because CHTS and SHTS are computed similarly as in the T_H transform (i.e. the secrets are the hashed seed and ciphertext) and thus resist to 1 adversarial decapsulation query. Then, dHS is used only once a MAC has been verified, which implies that an adversary relaying a correct tag should already know the root key HS. $\square$

Similarly, one can prove the security of the modified TLS 1.3 PSK-(EC)DHE 0-RTT handshake. Note that in our case the key-exchange will be done with KEMs, but we keep the “-(EC)DHE” in the name for consistency with the original protocol. We state this in the following informal theorem.

TLS 1.3 with KEM Handshake

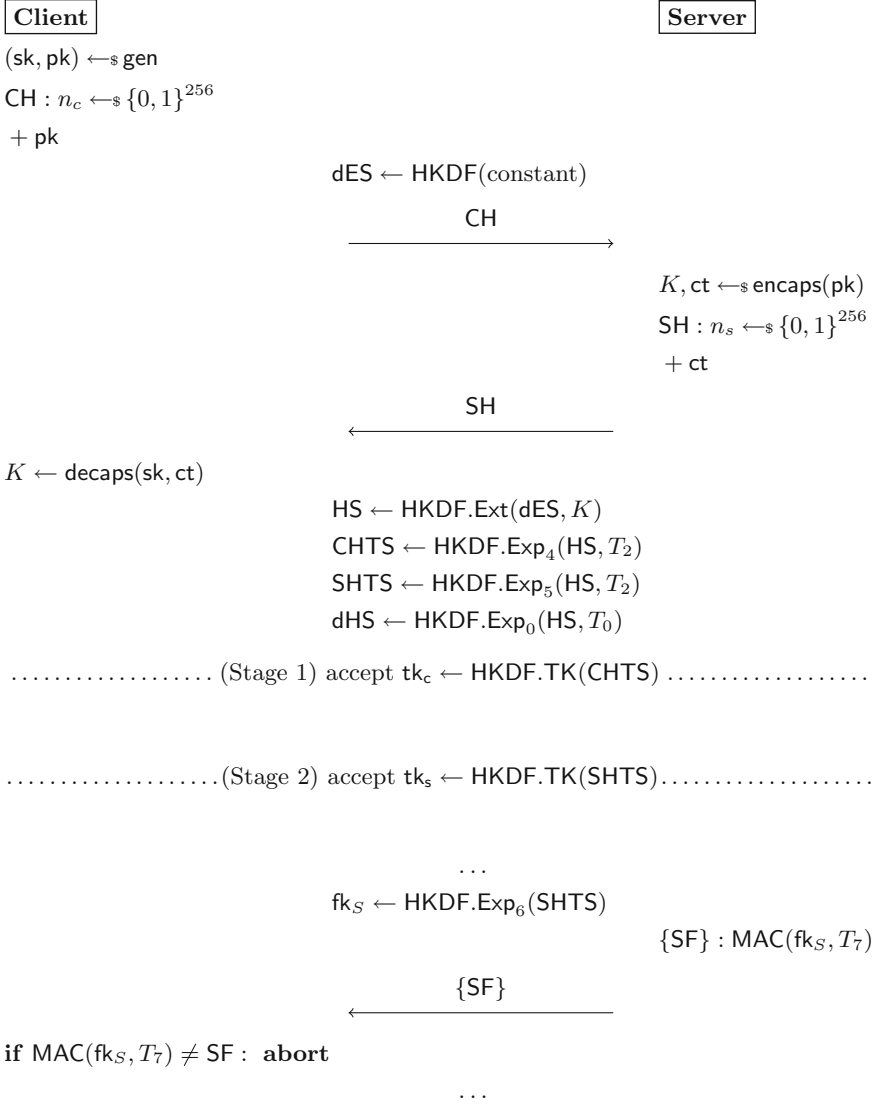


Fig. 14. TLS 1.3 handshake with KEM. $\{\dots\}$ indicates an encrypted message with tk_5 , T_i is the hash of the transcript up to message i . For simplicity, the **CH** (resp. **SH**) message captures both the *ClientHello* and *ClientKeyShare* (resp. *ServerHello* and *ServerKeyShare*). Only the relevant steps for the proof are shown. Keys in the remaining stages (3-6, not shown) are all derived from **dHS**.

Theorem 6. *The modified TLS 1.3 handshake in the pre-shared key (optional) 0-RTT mode with key-exchange (i.e. TLS 1.3 PSK-(EC)-DHE 0-RTT) is secure in the MultiStage model if the underlying KEM is OW-CPA (and signature, MAC, etc. are secure), in the sense of Dowling et al. [12].*

Proof. The only step in the original proof involving the KEMs can be dealt with a similar reduction from IND-1CCA-MAC as in the proof of Theorem 5. $\square$

Corollary 3. *The original TLS 1.3 handshake is MultiStage secure in the ROM if the CDH problem is hard (and the signature, MAC, etc. are secure). Stronger assumptions used in previous proofs (e.g. PRF-ODH [12]) are not necessary.*

Proof. This simply follows from the fact that DH can be described as a KEM $(\text{sk}, \text{pk}) := (x, g^x)$, $(K, \text{ct}) := (\text{pk}^y, g^y)$ and $\text{decaps}(\text{sk}, \text{ct}) := \text{ct}^x$. Integrating this KEM in our modified TLS 1.3 handshake exactly holds the standard TLS 1.3 handshake. Finally, this KEM is OW-CPA as long as the CDH problem is hard, thus by Theorems 4 and 5, the handshake is secure. One can also directly show that DH as used in TLS 1.3 is a IND-1CCA KEM. We provide such a proof in the full version [20]. $\square$

Remarks. Note that due to non-tightness of the bound in Theorem 4, the overall bound for TLS security is very much non-tight. This is clearly not sufficient to guarantee security in practice, and we leave as an interesting open question the improvement of the bounds. In addition, we leave security in the QROM as future work. As we extensively use the programming property of ROs, new QROM techniques such as the *compressed oracles* by Zhandry [33] might be of use in such a proof.

5 Impact

The transforms introduced in Sect. 3 produce IND-qCCA KEMs without any derandomization and re-encryption steps. Thus, using IND-1CCA ephemeral KEMs obtained through these transforms could speed up the decapsulation process in several protocols.

KEMTLS. As discussed in the introduction, improving the KEMTLS protocol [29] was the main motivation of this work. In particular, a more efficient decapsulation in the ephemeral KEM would decrease overall latency and computation on the client-side. In particular, this could be of interest for less powerful clients like IoT devices, which would not need to perform re-encryption. Overall, the efficiency gain in practice would obviously depend on the ephemeral KEM used, as encryption is expensive in some schemes while it is not in others. For instance, using KEMTLS with a modified version of SIKE (i.e. obtained through our transform instead of the FO one) would reduce probably significantly the handshake latency and computation cost on the client-side.

The same remarks apply to the very recent variants of KEMTLS with pre-distributed keys proposed by Günther et al. [18] and Schwabe et al. [31].

Note also that following a similar proof as the one in Sect. 4, we conjecture that one should be able to prove that CPA-security of the ephemeral KEM should suffice for KEMTLS to be secure in the ROM (but at the expense of a non-tight security bound, as in the TLS case).

TLS 1.3. TLS 1.3 only supports ephemeral DH as a key-exchange. In turn, in the original security proof [12], the snPRF-ODH assumption is used for the key-exchange security. The snPRF-ODH assumption can be seen as a variant of the hashed Diffie-Hellman assumption with a 1-time “decapsulation” oracle. More precisely, an adversary is given (g, g^u, g^v) and either $y_0 := \text{PRF}(g^{uv}, \text{ad}^*)$ or a random y_1 , where ad^* is some auxiliary data chosen by the adversary. Then, the adversary must distinguish between y_0 and y_1 with the help of *one* query to an oracle $\mathcal{O}((x, \text{ad}) \neq (g^u, \text{ad}^*)) := \text{PRF}(x^v, \text{ad})$.

One can notice that snPRF-ODH security is very close to IND-1CCA security transposed to DH key-exchange. Actually, one can show that IND-1CCA KEM is sufficient for the PQ TLS 1.3 handshake to hold. Indeed, instead of using our IND-1CCA-MAC assumption in the proof, one can use the decapsulation oracle of the IND-1CCA adversary to recover the key if needed. One can check the transition between games *B.1* and *B.2* in the proof of KEMTLS security [29] for more details.

Therefore, using IND-1CCA KEMs in the PQ TLS 1.3 handshake seems a sound idea, as in this case the security bound will offer better guarantees than with a OW-CPA KEM. In addition, the handshake would be faster using IND-1CCA KEMs generated by our transforms instead of the slower IND-CCA KEMs derived with FO.

Finally, by Corollary 3, we now know that the snPRF-ODH assumption is not necessary in the ROM for TLS 1.3 to be secure (even though the security bound is very much non-tight), but CDH is sufficient. Alternatively, as shown in the full version [20], DH as used in TLS 1.3 is actually an IND-1CCA KEM ($\approx$ snPRF-ODH) in the ROM if CDH holds. This gives a tighter security bound compared to Corollary 3.

Ratcheting. IND-1CCA security is also a property used (often implicitly) in several works on ratcheting. For instance, Jost et al. [22] build a *healable and key-updating public-key encryption* scheme based on a *one time* IND-CCA2 PKE (with authenticated data). The latter primitive can easily be made out of an IND-1CCA KEM using KEM/DEM techniques. Another paper by Poettering et al. [27] introduces a construction of *unidirectional ratcheted key exchange* (URKE) that is based (implicitly) on IND-1CCA KEMs, as noticed by Balli et al. [2].

In another recent paper, Brendel et al. [6] propose an alternative to the Signal handshake based on KEMs and designated verifier signature schemes. They first define a core protocol that uses two KEMs in the same vein as KEMTLS: one with long-term keys for implicit authentication of one of the parties and another one with ephemeral keys for guaranteeing forward security. Again, the latter one requires only IND-1CCA security for the handshake to be secure. Similarly, in

the full Signal-like handshake built upon the core protocol (called SPQR), three KEMs are used and one requires only IND-1CCA security.

Concerns Over Key-reuse. The main security risk of using an IND-1CCA KEM instead of its IND-CCA counterpart is the vulnerability to key-reuse/misuse attacks. Indeed, if a system/protocol is misimplemented s.t. the IND-1CCA KEM is used with a “static” public key instead of an ephemeral one, an adversary might be able to recover the secret key after several decryption queries. In KEMTLS, this risk is mitigated by the use of an IND-CCA KEM in addition to the ephemeral one (which can be IND-1CCA). In particular, the final shared key is derived from shares of both KEMs. Thus, even if the public-key meant to be ephemeral is reused, the final shared key should remain “secure” (but forward security would be lost).

In other systems (e.g. TLS 1.3), the risk of key recovery after a few reuses could be mitigated by using hybrid cryptography. For instance, a very efficient IND-CCA KEM could be combined with an IND-1CCA one. That would improve the overall security and resistance against key-reuse attacks at a small cost (see e.g. Giacon et al. [16] or Bindel et al. [3] for KEM combiners). Finally, we stress again that if ephemeral keys were misimplemented as static ones in these systems, the forward security property would be lost.

Conclusion. Ratcheting and several recent protocols (e.g. TLS 1.3) are aiming at forward security, which often implies generating a new pair of public/secret keys for each message exchanged. Informally, in many settings this means that an adversary requesting a decryption will be able to do so only once for a given key pair. Thus, IND-1CCA security of the underlying encryption/encapsulation primitive might be sufficient to guarantee the security of such systems.

Acknowledgment. We thank Daniel Collins for pointing out possible use-cases of IND-1CCA KEMs in ratcheting and the anonymous reviewers for their helpful comments. Lo s Huguenin-Dumittan is supported by a grant (project N  192364) of the Swiss National Science Foundation (SNSF).

References

1. OQS OpenSSL (June 2021). <https://github.com/open-quantum-safe/openssl>
2. Balli, F., R sler, P., Vaudenay, S.: Determining the core primitive for optimally secure ratcheting. In: Moriai, S., Wang, H. (eds.) ASIACRYPT 2020. LNCS, vol. 12493, pp. 621–650. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-64840-4_21
3. Bindel, N., Brendel, J., Fischlin, M., Goncalves, B., Stebila, D.: Hybrid key encapsulation mechanisms and authenticated key exchange. In: Ding, J., Steinwandt, R. (eds.) PQCrypto 2019. LNCS, vol. 11505, pp. 206–226. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-25510-7_12
4. Bindel, N., Hamburg, M., H velmanns, K., H lsing, A., Persichetti, E.: Tighter proofs of CCA security in the quantum random oracle model. In: Hofheinz, D., Rosen, A. (eds.) TCC 2019. LNCS, vol. 11892, pp. 61–90. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-36033-7_3

5. Brendel, J., Fischlin, M., Günther, F., Janson, C.: PRF-ODH: Relations, instantiations, and impossibility results. Cryptology ePrint Archive, Report 2017/517 (2017). <https://ia.cr/2017/517>
6. Brendel, J., Fiedler, R., Günther, F., Janson, C., Stebila, D.: Post-quantum asynchronous deniable key exchange and the signal handshake. Cryptology ePrint Archive, Report 2021/769 (2021). <https://eprint.iacr.org/2021/769>
7. Celi, S., et al.: Implementing and measuring KEMTLS. Cryptology ePrint Archive, Report 2021/1019 (2021). <https://ia.cr/2021/1019>
8. Cramer, R., et al.: Bounded CCA2-secure encryption. In: Kurosawa, K. (ed.) ASIACRYPT 2007. LNCS, vol. 4833, pp. 502–518. Springer, Heidelberg (2007). https://doi.org/10.1007/978-3-540-76900-2_31
9. Crockett, E., Paquin, C., Stebila, D.: Prototyping post-quantum and hybrid key exchange and authentication in TLS and SSH. IACR Cryptol. ePrint Arch. **2019**, 858 (2019)
10. Davis, H., Günther, F.: Tighter proofs for the SIGMA and TLS 1.3 key exchange protocols. In: Sako, K., Tippenhauer, N.O. (eds.) ACNS 2021. LNCS, vol. 12727, pp. 448–479. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-78375-4_18
11. Diemert, D., Jager, T.: On the tight security of TLS 1.3: theoretically sound cryptographic parameters for real-world deployments. J. Cryptol. **34**(3), 1–57 (2021)
12. Dowling, B., Fischlin, M., Günther, F., Stebila, D.: A cryptographic analysis of the TLS 1.3 handshake protocol. J. Cryptol. **34** (2020)
13. Fluhrer, S.: Cryptanalysis of ring-LWE based key exchange with key share reuse. Cryptology ePrint Archive, Report 2016/085 (2016). <https://eprint.iacr.org/2016/085>
14. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. In: Wiener, M. (ed.) CRYPTO 1999. LNCS, vol. 1666, pp. 537–554. Springer, Heidelberg (1999). https://doi.org/10.1007/3-540-48405-1_34
15. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. J. Cryptol. **26**(1), 80–101 (2013). <https://doi.org/10.1007/s00145-011-9114-1>
16. Giacon, F., Heuer, F., Poettering, B.: KEM Combiners. Cryptology ePrint Archive, Report 2018/024 (2018). <https://eprint.iacr.org/2018/024>
17. Günther, F.: Modeling advanced security aspects of key exchange and secure channel protocols (2018)
18. Günther, F., Towa, P.: KEMTLS with delayed forward identity protection in (almost) a single round trip. Cryptology ePrint Archive, Report 2021/725 (2021). <https://eprint.iacr.org/2021/725>
19. Hofheinz, D., Hövelmanns, K., Kiltz, E.: A modular analysis of the Fujisaki-Okamoto transformation. In: Kalai, Y., Reyzin, L. (eds.) TCC 2017. LNCS, vol. 10677, pp. 341–371. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-70500-2_12
20. Huguenin-Dumittan, L., Vaudenay, S.: A note on IND-qCCA security in the ROM and its applications: CPA security is sufficient for TLS 1.3. Cryptology ePrint Archive, Report 2021/844 (2021). <https://ia.cr/2021/844>
21. Jao, D., et al.: SIKE (2020). <https://csrc.nist.gov/Projects/post-quantum-cryptography/round-3-submissions>
22. Jost, D., Maurer, U., Mularczyk, M.: Efficient ratcheting: almost-optimal guarantees for secure messaging. In: Ishai, Y., Rijmen, V. (eds.) EUROCRYPT 2019. LNCS, vol. 11476, pp. 159–188. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-17653-2_6

23. Kuchta, V., Sakzad, A., Stehlé, D., Steinfeld, R., Sun, S.-F.: Measure-rewind-measure: tighter quantum random oracle proofs for One-Way to hiding and CCA security. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020. LNCS, vol. 12107, pp. 703–728. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-45727-3_24
24. Okamoto, T., Pointcheval, D.: REACT: rapid enhanced-security asymmetric cryptosystem transform. In: Naccache, D. (ed.) CT-RSA 2001. LNCS, vol. 2020, pp. 159–174. Springer, Heidelberg (2000). https://doi.org/10.1007/3-540-45353-9_13
25. Paquin, C., Stebila, D., Tamvada, G.: Benchmarking post-quantum cryptography in TLS. Cryptology ePrint Archive, Report 2019/1447 (2019). <https://eprint.iacr.org/2019/1447>
26. Pereira, M., Dowsley, R., Hanaoka, G., Nascimento, A.C.A.: Public key encryption schemes with bounded CCA security and optimal ciphertext length based on the CDH assumption. In: Burmester, M., Tsudik, G., Magliveras, S., Ilić, I. (eds.) ISC 2010. LNCS, vol. 6531, pp. 299–306. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-18178-8_26
27. Poettering, B., Rösler, P.: Towards bidirectional ratcheted key exchange. In: Shacham, H., Boldyreva, A. (eds.) CRYPTO 2018. LNCS, vol. 10991, pp. 3–32. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-96884-1_1
28. Saito, T., Xagawa, K., Yamakawa, T.: Tightly-secure key-encapsulation mechanism in the quantum random oracle model. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018. LNCS, vol. 10822, pp. 520–551. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-78372-7_17
29. Schwabe, P., Stebila, D., Wiggers, T.: Post-quantum TLS without handshake signatures. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, pp. 1461–1480 (2020)
30. Schwabe, P., Stebila, D., Wiggers, T.: Post-quantum TLS without handshake signatures. Cryptology ePrint Archive, Report 2020/534 (2020). <https://eprint.iacr.org/2020/534>
31. Schwabe, P., Stebila, D., Wiggers, T.: More efficient post-quantum KEMTLS with pre-distributed public keys. Cryptology ePrint Archive, Report 2021/779 (2021). <https://eprint.iacr.org/2021/779>
32. Yamakawa, T., Yamada, S., Matsuda, T., Hanaoka, G., Kunihiro, N.: Reducing public key sizes in bounded CCA-secure KEMs with optimal ciphertext length. In: Desmedt, Y. (ed.) ISC 2013. LNCS, vol. 7807, pp. 100–109. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-27659-5_7
33. Zhandry, M.: How to record quantum queries, and applications to quantum indistinguishability. In: Boldyreva, A., Micciancio, D. (eds.) CRYPTO 2019. LNCS, vol. 11693, pp. 239–268. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-26951-7_9